

Execution Audit: TP loss, dual-device CSRF, and drilldown 500 — root causes and fixes

Five specialized auditors read the executor, trade manager, MT5 REST bridge, auth layer, API, and database code in parallel; every critical or high finding was then independently re-verified against the code by an adversarial agent instructed to refute it. All three reported symptoms are fully explained with exact code paths. Nothing below is speculative — each finding cites the lines that prove it.

6

critical (verified)

7

high (verified)

20

medium / low

0

findings refuted

3 / 3

symptoms root-caused

SECTION 1

Symptom 1 — trade executes but TP never attaches, position rides to SL

This is not one bug. Four independent, verified mechanisms all end in the same state: a live position with `tp=0` that no background process will ever repair. The common thread is the orphan protective-TP watchdog's sibling exclusion (`db.py:1134–1138`): as soon as a signal has stage ≥ 2 rows, the watchdog is *permanently* forbidden from touching stage 1 — so any failure in the one-shot TP attach path is terminal.

CRITICAL trade_manager.py:488–503 → executor.py:1026–1041

1.1 Correlated follow-up bands are persisted without `signal_sl` / `signal_tp`

`_handle_correlated_followup` builds the stage 2..N band rows without the `signal_sl` / `signal_tp` keys, so `db.create_staged_entries` inserts NULL for both (the direct-zone path at `trade_manager.py:710–711` and text-only path at `329–330` both set them — this insert site was simply missed; the EXEC2-01 comment at `executor.py:616` even claims the row "carries the follow-up's real SL/TP"). Bands that are in-zone at arrival fire with the real TP, but any band that *arms* and later fires through the zone watcher reads the NULLs: it opens with `tp=0.0` and a default-pips SL measured off the *band* edge instead of the signal's SL. Second consequence: the same NULLs make the TP/SL-reached cascade fall back to the orphan's signals row (logged with `sl=0/tp=0`), so `executor.py:644` skips the cascade and **remaining stages keep firing after price already hit the real TP or SL**.

FIX — add `"signal_sl": signal.sl, "signal_tp": signal.target_tp` to the rows dict, mirroring `trade_manager.py:710–711`; on pairing, also update the orphan's signal targets so the cascade fallback works.

CRITICAL trade_manager.py:418–425 → server.py:372

1.2 A follow-up without a TP sends `tp=0.0` — which MT5 interprets as "remove the TP"

When the follow-up has no numeric TP (e.g. "TP: open"), `new_tp` stays `0.0` and is sent in the SLTP modify anyway. The bridge treats `0.0` as an explicit value (`req.tp if req.tp is not None else pos.tp`), so any existing TP is erased. And because the pairing just created sibling band rows, the watchdog exclusion (1.1) guarantees no protective TP is ever re-attached. Verified end state: stage-1 rides SL-only to stop-out.

FIX — `new_tp = calculate_tp_with_jitter(...)` if `signal.target_tp` else `None`; the server's `is not None` guard then preserves the existing TP. Broaden the watchdog to any bot-owned position with `tp==0` older than the correlation window, not only sibling-free stage-1s.

CRITICAL `trade_manager.py:423–472, server.py:379–380`

1.3 The stage-1 TP attach is single-shot with zero post-trade verification

The *only* mechanism that attaches the follow-up's TP to an already-open text-only position is one `modify_position` call. Broker rejects (retcode 10016 invalid-stops — likely exactly when price has run toward the TP — or 10025) become HTTP 400 and get zero retries; transport timeouts get 3 then give up. The failure is logged, appended as `stage1_align_failed ...` which `notifier.py` has no case for, so it's silently omitted from your Telegram execution message. No retry queue, no periodic re-check, no verification that the live position actually carries the requested SL/TP. One failed modify → position unprotected forever (the sibling rows block the watchdog). The modify also bundles SL+TP in one `TRADE_ACTION_SLTP` request, so a TP rejected for stops-level throws away the SL update too.

FIX — persist the desired SL/TP onto the stage row at pairing time, then add a reconciliation sweep to the existing 10s zone-watch tick: any filled stage whose live position deviates from persisted targets gets a re-issued modify with bounded retries and an operator alert after N failures. Split SL and TP modifies (or fall back to SL-only) when TP is rejected. Add a `stage1_align_failed` case to the notifier.

CRITICAL (MULTI-ACCOUNT) / LATENT (SINGLE-ACCOUNT)

`executor.py:107–157, signal_correlator.py:69–79`

1.4 Multi-account correlation pairs follow-ups to the wrong account's signal

Each account runs in a temp per-account TradeManager, so one text-only Telegram message logs N distinct `signal_id`s and registers N orphans in the *shared* correlator — keyed only by (symbol, direction), popped LIFO. Accounts are shuffled independently for the text-only and the follow-up message, so a fully correct pairing requires the second shuffle to be the exact reverse of the first: probability 1/n! (50% failure with just 2 accounts). A mispair is symmetric — *both* accounts silently skip stage-1 alignment, both are excluded from the watchdog, the D-16 cascade wrongly cancels the other account's armed bands, and the duplicate-direction check blocks band fills. *Caveat from verification*: the checked-in `accounts.json` has one enabled account, where this can't fire — but it makes the advertised multi-account mode unsafe to ever enable, and `accounts.example.json` ships two.

FIX — give one Telegram message one canonical `signal_id`: log the signal once in `Executor.execute_signal` before the fan-out and pass it down. Key the correlator by (account, symbol, direction) as defense-in-depth, and add `account_name` predicates to the sibling NOT EXISTS in `get_orphan_candidate_stage1s` and to `get_stage_by_comment`.

Which one is biting you today? On a single-account deployment, 1.1 is the most probable live cause (any signal whose later stages fire via the zone watcher opens TP-less), with 1.2 and 1.3 covering follow-up-driven cases. All three are cheap fixes. 1.4 becomes mandatory before you ever enable a second account.

SECTION 2

Symptom 2 — laptop + phone: one device works, the other gets "csrf token invalid"

The backend fully supports concurrent devices — there is no global token, no rotation, no server-side session singleton. Sessions are stateless signed cookies and the CSRF check is a stateless per-request double-submit compare. The bug is a cookie lifecycle mismatch with no recovery path.

HIGH `api/auth.py:80–87, 169–176` · `dashboard.py:183` · `LoginView.tsx:42`

2.1 CSRF cookie dies with the browser; the session cookie lives 30 days; the SPA never reseeds

The `telebot_csrf` cookie is set *without* `max_age` — a browser-session cookie — while `telebot_session` persists 30 rolling days. When one device's browser restarts (or the mobile OS evicts session cookies, which iOS/Android do aggressively), that device still auto-authenticates via the persistent session cookie and renders the app shell — but `document.cookie` no longer has a CSRF token. The SPA seeds CSRF *only* in the login view (sole `seedCsrf()` call site), which never mounts for an authenticated user. Every mutation then sends an empty `X-CSRF-Token` → 403 "CSRF token invalid", and the error handler only auto-recovers from 401. The device is stuck until manual re-login — while the device whose browser kept running works fine. Same defect also breaks a *single* device after any browser restart.

FIX — three coordinated changes, any one of which resolves the symptom: (1) set `max_age=30*24*60*60` on both CSRF `set_cookie` calls; (2) on app boot after `/auth/me` succeeds, call `seedCsrf()` when the cookie is absent; (3) in `http.ts`, on a 403 "CSRF token invalid", fetch `/auth/csrf` once and retry once. Do (1)+(2) for robustness.

Secondary auth findings (not the symptom, worth queueing): non-revocable rolling 30-day sessions with client-only logout (medium), CSRF-exempt `GET /logout` allows cross-site forced logout (low), unprefixed double-submit cookie (low), login lockout keyed on spoofable

X-Real-IP (low). Positives worth keeping: timing-safe compares, argon2id with rate-limit before hash cost, correct `httponly=False` on the double-submit cookie.

SECTION 3

Symptom 3 — 500 "unable to get..." on active-trade details

HIGH `db.py:1284, 1291 · api/positions.py:62`

3.1 Drilldown crashes on every staged trade: JSONB arrives as `str`, code calls `.get()` on it

The pool is created with plain `asyncpg.create_pool` and no JSON type codec, so JSONB columns come back as raw JSON *strings*. `get_position_drilldown` does `settings = stage["snapshot_settings"]` or `{}` — a non-empty string is truthy — then `settings.get("default_sl_pips")` → `AttributeError: 'str' object has no attribute 'get'`. The codebase itself proves the runtime type: `executor.py:898` and `dashboard.py:246` both `isinstance`-guard this exact column; this is the one un-guarded reader. It fires for precisely the trades with `staged_entries` rows joined on ticket — every staged/zone fill — while plain single-stage trades take the crash-free fallback, which is why only *some* details 500. And since `api/errors.py` installs no catch-all handler, the error escapes un-enveloped and the SPA shows its generic failure text.

FIX — register the codec once at pool init: `create_pool(..., init=lambda c: c.set_type_codec('jsonb', encoder=json.dumps, decoder=json.loads, schema='pg_catalog'))`, then delete the scattered `isinstance` guards. Add a catch-all exception handler so unexpected errors return the standard error envelope.

Related drilldown polish: the payload omits every `*_display` twin the SPA renders (fill history shows em-dashes), and `sl_at_fill` mixes units — pips for staged fills, absolute price for single-stage fills.

SECTION 4

Other verified execution risks (money-losing paths beyond the symptoms)

CRITICAL `mt5_connector.py:586–619`

4.1 Blind retry of `POST /api/v1/order` on `ReadTimeout` — duplicate market orders

`_request` retries every request 3x on `ConnectError`, `ConnectTimeout` and `ReadTimeout`, with no method/path discrimination. A `ReadTimeout` means the order may already have filled (the bridge runs MT5 calls synchronously on its event loop, so a 15s client timeout while the loop is busy is realistic) — the retry then doubles your exposure with the same comment. Reconciliation can't even detect it afterwards: `_sync_positions` collapses positions into a by-comment dict, so duplicates merge into one entry. Conversely, if all 3 attempts time out, the trade is recorded *failed* while a real position sits on the broker untracked — and the reconnect reconcile only adopts stages in awaiting states, never `failed` ones. The unstaged path is worse: it uses the constant comment `"telebot"`, so it has no idempotency probe at all.

FIX — retry only GETs and connection-level failures (request provably never left). On `ReadTimeout` of an order `POST`, probe positions by the unique comment before any resend and adopt the position if found; or make the bridge idempotent — reject/return-existing for a comment it already executed. Give unstaged orders unique comments first.

CRITICAL `trade_manager.py:715`, `executor.py:533`, `db.py:1055–1062`

4.2 Duplicate stage fire: at-arrival firing races the 10s zone-watch poll with no row claim

Band rows are inserted as `awaiting_zone` and then fired in the same coroutine, flipping to `filled` only after the order returns. The independent zone-watch loop polls `status='awaiting_zone'` with no `FOR UPDATE`, no claim status, no compare-and-set. A tick landing inside the fire window submits the same stage twice — the idempotency probe passes for both because the first order hasn't landed on the broker yet. `update_stage_status` has no status precondition, so last-writer-wins and one broker position ends up untracked. Result: 2x intended lot exposure.

FIX — atomic claim before submit: `UPDATE staged_entries SET status='firing' WHERE id=$1 AND status='awaiting_zone' RETURNING id`; submit only on claim success, in both fire paths. Add status preconditions to `update_stage_status` so terminal states (incl. kill-switch cancellations) can never be overwritten by in-flight fills.

HIGH mt5-rest-server/server.py:70-86

4.3 REST bridge runs blocking MT5 C calls directly on the asyncio event loop

`_run` is `async def` but calls the MT5 function inline. While one `order_send / history_deals_get` blocks, *everything* — `/ping`, `/price`, `/positions` — freezes. This feeds the ReadTimeout retries in 4.1, and sustained blockage (~45s across ping retries) makes the heartbeat declare the connection dead and re-login — the exact trigger for your known retcode-10027 "AutoTrading disabled on account change" cascade.

FIX — a module-level `ThreadPoolExecutor(max_workers=1)` and `await loop.run_in_executor(...)` in `_run`: single worker keeps MT5 calls serialized (the lib isn't thread-safe) while the loop stays responsive. Move the paired `mt5.last_error()` calls into the same executor job so they read the right error.

HIGH mt5-rest-server/server.py:415

4.4 Close path hardcodes `ORDER_FILLING_IOC` that the open path negotiates around

`create_order` carefully negotiates the filling mode from `symbol_info` (with a comment noting brokers like Vantage reject IOC), but `close_position` hardcodes IOC. On any symbol where the open path fell back to FOK or RETURN, every bot-initiated close — including the **kill-switch flatten** — fails with retcode 10030: positions the bot can open but never close.

FIX — extract the negotiation into a helper used by both paths; optionally retry a close once with RETURN on 10030.

HIGH executor.py:104-134

4.5 Kill-switch race: pause flag checked once, before up to (N-1)×5s of stagger

`execute_signal` reads `_trading_paused` only at entry. Hit the kill switch mid-stagger and the in-flight signal keeps opening positions on remaining accounts *after* `emergency_close` flattened everything — you believe you're flat while carrying live exposure. The zone-watch loop already has the correct D-21 mid-tick re-check; the primary path doesn't.

FIX — re-check the flag after each stagger sleep, before `_execute_single_account`, and break with a "skipped: kill switch" result.

HIGH `api/actions.py:84, db.py:363`

4.6 Dashboard full-close permanently records pnl = 0.0

The close route writes `update_trade_close(..., 0.0, ...)` as a "placeholder" — but the history-sync loop only scans `status='opened'` rows, so once the route flips the row to `closed` the real deal profit is never backfilled. Every dashboard-closed trade shows \$0 in history and silently corrupts win-rate, profit-factor and net-P&L analytics forever.

FIX — don't mark closed from the route: leave the row in a status history-sync scans (e.g. `closing`, added to `get_open_trade_tickets_for_account`) and let the sync write the authoritative `deal.profit`; or fetch the closing deal immediately after the close and record its profit.

MEDIUM `mt5-rest-server/server.py:340`

4.7 `DONE_PARTIAL` treated as total failure; no retcode taxonomy; deviation hardcoded

A partial fill (10010) returns `success=False, ticket=0`: the bot logs a failed trade while a live partial position exists — no trades row, wrong risk accounting, invisible in the dashboard (it does keep its SL/TP, and symbol-scan management still reaches it). Transient retcodes (10004 requote, 10021, 10006) are never retried; terminal ones aren't distinguished; slippage is a hardcoded `deviation: 20`.

FIX — on 10010 return success with the actual filled volume/ticket. Classify retcodes retryable-vs-terminal (bounded retries with fresh price for requotes). Make deviation a per-request config value.

SECTION 5

Industry-standards gap analysis

Benchmarked against what a professional order-execution system does. The staged-entry design has real strengths — pre-submit intent rows, comment-based idempotency probes, reconnect reconcile, kill-switch drain — but the gaps cluster exactly where money is lost.

STANDARD PRACTICE

TELEBOT TODAY

GAP

Post-trade verification	Fire-and-forget: nothing confirms SL/TP landed after <code>order_send</code> /modify	Root of symptom 1 (§1.3); the orphan watchdog covers only sibling-free stage-1s
Idempotent order submission	Blind 3× retry incl. order POSTs; no client order-id enforced server-side; unstaged comment not unique	Double fills, untracked positions (§4.1)
Durable intent + reconciliation loop	Staged path writes intent rows; unstaged path has none; reconcile runs only on heartbeat reconnect, never at process startup	Crash leaves phantom armed stages and orphan broker positions
Retryable vs terminal error taxonomy	Any retcode ≠ DONE → failure; transport errors retried indiscriminately	Requotes never retried; partial fills discarded (§4.7)
Kill-switch semantics	Checked once per signal; close path can't fill on FOK/RETURN symbols	Exposure after flatten (§4.5); flatten can fail entirely (§4.4)
Concurrency control on shared state	No row locks, no compare-and-set, no status preconditions anywhere in the stage state machine	Duplicate fires, resurrection of cancelled stages (§4.2)
Non-blocking broker gateway	MT5 C calls inline on the event loop	Self-inflicted timeouts, false reconnects, the 10027 cascade (§4.3)
Precision handling	Floats and hardcoded rounding; no broker <code>volume_step</code> / <code>digits</code> awareness	Occasional broker rejects; fixed-lot stage floor can exceed configured total

SECTION 6

Medium & low findings (deduplicated)

SEV	WHERE	FINDING
-----	-------	---------

MED	trade_manager.py:415	SL-less correlated follow-up raises <code>TypeError</code> before band creation — the D2-13 guard exists only in <code>_handle_open</code>
MED	trade_manager.py:1269	Filled-limit detection in <code>cleanup_expired_orders</code> matches <code>str(ticket)</code> against comments that never contain tickets — structurally always false; filled-then-expired limits are recorded as cancelled
MED	trade_manager.py:1014	Limit-order trades logged <code>status='pending'</code> never transition — broker-side closes of limit-filled positions are never reconciled
MED	bot.py:277	No startup reconcile: the D-24 reconcile runs only on heartbeat reconnect — a crash leaves phantom armed stages
MED	db.py:1055	<code>update_stage_status('filled')</code> has no status precondition — kill-switch-cancelled stages can be resurrected by in-flight fills
MED	executor.py:991	Zone-watch snapshot rebuild requires every <code>AccountSettings</code> field — schema drift silently downgrades <code>fixed_lot</code> stages to percent sizing
MED	executor.py:751	Point-band stages (<code>zone_low == zone_high</code>) can never pass the zone-watch pre-flight — tolerance degenerates to zero
MED	dashboard.py:179	Sessions are non-revocable signed cookies, rolling 30 days; logout is client-side only
MED	api/positions.py:66	Drilldown payload omits all <code>*_display</code> twins — SPA fill history renders em-dashes
MED	trade_manager.py:1004	Unstaged path has no durable intent row before submission and no forward reconciliation
LOW	trade_manager.py:1107	MODIFY_SL/TP and CLOSE signals act on <i>all</i> symbol positions incl. manual ones — no magic/comment filter; retcode 10025 no-change reported as failure
LOW	trade_manager.py:794	Daily trade-limit check is read-then-act across concurrent handlers — limit can be exceeded
LOW	risk_calculator.py:64	Float arithmetic throughout; no <code>volume_step / digits</code> awareness; fixed-lot per-stage floor can exceed configured total

LOW	mt5_connector.py:737	Structured broker error detail discarded at connector boundary — failures surface as generic "REST API request failed"
LOW	api/errors.py:83	No catch-all /api/v2 exception handler — unhandled errors bypass the error envelope
LOW	db.py:1291	sl_at_fill mixes units: pips for staged fills, absolute price for single-stage
LOW	dashboard.py:215	GET /logout is CSRF-exempt — cross-site forced logout
LOW	api/deps.py:62	Double-submit cookie without __Host- prefix or session binding
LOW	dashboard.py:117	Login logout keyed on unconditionally-trusted X-Real-IP

SECTION 7

Remediation plan

Ordered by money-at-risk per line of code changed. P0 items are all small, surgical diffs; each maps to a finding above.

P0 — stop the bleeding (small diffs, do first)

- §1.1** Add `signal_sl / signal_tp` to the correlated follow-up rows dict (`trade_manager.py:488`) — a two-line change that removes the largest single-account TP-loss path.
- §1.2** Send `tp=None` instead of `0.0` when the follow-up has no TP (`trade_manager.py:418`).
- §3.1** Register the JSONB codec at pool init (`db.py:60`) — fixes the drilldown 500 everywhere at once; remove the scattered isinstance guards afterwards.
- §2.1** Add `max_age` to both CSRF `set_cookie` calls + reseed on app boot — fixes dual-device management.
- §4.1** Stop retrying order-placing POSTs on ReadTimeout; probe by comment before any resend.
- §4.5** Re-check the kill switch after each stagger sleep.

P1 — structural correctness (this sprint)

1. **§1.3** Persist target SL/TP per stage and add the 10s verification sweep: live position deviates from targets → re-modify with bounded retries → alert. This converts every remaining one-shot failure mode into a self-healing one.
2. **§4.2** Atomic `status='firing'` claim before stage submission + status preconditions in `update_stage_status` .
3. **§4.4** Share the filling-mode negotiation with `close_position` ; verify the kill switch can actually flatten on your broker's symbols.
4. **§4.3** Move MT5 calls onto a single-worker thread executor in the bridge.
5. **§4.6** Route dashboard closes through history-sync (a `closing` status) so real P&L is recorded.
6. **§1.4** One canonical `signal_id` per Telegram message, logged before fan-out — mandatory before enabling a second account.

P2 — hardening (queued)

1. Retcode taxonomy + partial-fill handling + configurable deviation (§4.7).
2. Startup reconcile in `bot.py` ; adopt unknown bot-comment positions; unique comments on the unstaged path.
3. Catch-all API error envelope; drilldown `*_display` fields; `sl_at_fill` unit fix.
4. Session revocation (server-side token version), POST-only logout, `__Host-` cookie prefix.
5. Magic-number filter for MODIFY/CLOSE handlers; atomic daily-limit check; Decimal/step-aware sizing.

Produced by a 23-agent audit (5 domain auditors, 18 adversarial verifiers) over the telebot repository on 2026-07-09. Every critical/high finding was independently re-verified against the code by an agent instructed to refute it; zero findings were refuted. Full machine-readable findings: `scratchpad/audit_findings.json` .

APPENDIX A

Full findings with evidence and adversarial verdicts

A.1 Domain summaries

Domain: exec-tp-sl

The TP/SL attachment symptom is real and has two independent, provable root causes plus a fragile single-shot attach path. (1) CRITICAL: `_handle_correlated_followup` creates band rows WITHOUT `signal_sl/signal_tp` (`trade_manager.py:488-503`), so every stage that arms and later fires via the executor's zone watch opens with `tp=0.0` and a default-pips SL off the band edge (`executor.py:1026-1041` -> `trade_manager.py:971`); the orphan protective-TP watchdog explicitly excludes stages with siblings (`db.py:1134-1138`), so nothing ever attaches a TP and the position rides to SL — the same NULLs also neuter the `tp_reached/sl_reached` cascade (`executor.py:644`), so stages keep firing after the real TP is hit. (2) CRITICAL: in multi-account operation, each account logs its OWN `signal_id` per message (executor fans out per-account temp `TradeManagers`, `executor.py:136/150-157`; `trade_manager.py:269`) while the correlator pairs only by (symbol,direction) LIFO under two independent random shuffles (`signal_correlator.py:69-79`, `executor.py:108`) — cross-account mispairing (~50% with 2 accounts) makes the stage-1 SL/TP alignment silently no-op (`trade_manager.py:407-414/473-477`), permanently disqualifies the position from the protective watchdog, falsely cascades the other account's stages, and blocks band fills via the duplicate-direction check. (3) HIGH: even when pairing is correct, the TP attach is a single `modify_position` with no retry; a REST timeout or broker invalid-stops (10016) reject (`server.py:379-380` -> HTTP 400 -> `OrderResult` failure) is logged and dropped, leaving the position TP-less forever. Additional execution-correctness defects: retrying `POST /api/v1/order` on `ReadTimeout` can double-execute market orders (`mt5_connector.py:608`); the follow-up path sends `tp=0.0` (TP removal) for 'TP: open' signals; SL-less follow-ups raise `TypeError` pre-band; server `close_position` hardcodes IOC filling while the open path negotiates it; filled-limit detection in `cleanup_expired_orders` is unreachable; zone-watch snapshot rebuild silently downgrades `fixed_lot` sizing on schema drift; and `modify/close` signal handlers touch manual positions with no magic/comment filter.

Domain: auth-csrf

Symptom 2 ("laptop + phone: only one device can manage trades, the other gets 'csrf token invalid'") is NOT caused by a global token, server-side session singleton, or login-rotation invalidating other devices — the backend is fully multi-device-capable: sessions are stateless Starlette signed cookies (dashboard.py:179-187) and the CSRF guard is a stateless per-request double-submit compare of the telebot_csrf cookie against the X-CSRF-Token header (api/deps.py:62-66). The proven root cause is a cookie-lifecycle mismatch with no recovery path: the telebot_csrf cookie is set WITHOUT max_age (api/auth.py:80-87, 169-176) so it dies with the browser process, while the telebot_session cookie persists 30 days (rolling). A device whose browser restarted or whose mobile OS evicted session cookies still auto-authenticates (App.tsx renders the shell straight off /auth/me 200) but has no CSRF cookie; the SPA only seeds CSRF in the login view (LoginView.tsx:41-45, sole seedCsrf call site), so http.ts:51-53 sends an empty X-CSRF-Token, every mutation 403s with "CSRF token invalid", and queryClient.ts:42-48 only auto-recovers from 401 — leaving that device permanently broken until manual re-login while the most-recently-used device keeps working. Fix: persist the CSRF cookie (max_age matching the session), reseed it on app boot when absent, and optionally retry-once-after-reseed on CSRF 403s. Secondary audit findings: non-revocable rolling 30-day sessions with client-only logout (medium), CSRF-exempt GET /logout enabling cross-site forced logout (low), unprefixed/unbound double-submit cookie (low), and blind trust of X-Real-IP for the login lockout key (low, mitigated by current docker/nginx topology). Positives: timing-safe compares throughout, argon2id with rate-limit ordered before the hash cost, config-driven Secure flag, and correct httponly=False on the double-submit cookie.

Domain: api-500

Symptom 3 is fully explained: the 500 comes from GET /api/v2/positions/{account}/{ticket} (api/positions.py:62 -> db.get_position_drilldown). The staged_entries.snapshot_settings JSONB column is returned by asyncpg as a raw JSON **string** (the pool at db.py:60-65 registers no type codec, and the value is written with json.dumps at db.py:1039), and db.py:1284/1291 does ``settings = stage["snapshot_settings"] or {}`` then ``settings.get("default_sl_pips")`` — `AttributeError: 'str' object has no attribute 'get'`. The crash fires for exactly the trades that have staged_entries rows joined on mt5_ticket (every staged/zone fill, since trade_manager.py:1003/1013 writes the same result.ticket to both tables), while text-only single-stage trades take the crash-free fallback at db.py:1296-1305 — matching the operator seeing it only on some active trades (the drilldown is only reachable from the open-positions view; closed trades go through /history). The codebase itself proves the str runtime type: executor.py:897-904 and dashboard.py:245-250 both isinstance-str-guard the identical column; get_position_drilldown is the one un-guarded reader. Because api/errors.py:85-87 installs no catch-all handler, the AttributeError surfaces as a bare un-enveloped 500 and the SPA ErrorPanel degrades to its generic "Request failed (HTTP 500)" text — the reported "unable to get ...". The other detail/list endpoints in this domain (positions list, /stages, /signals, /history, /analytics) are None-guarded or route the same JSONB through the guarded dashboard._enrich_stage_for_ui, so they do not share the 500; the remaining defects found are a drilldown payload that omits every *_display twin the SPA renders (fill history shows all em-dashes), an sl_at_fill pips-vs-price unit mix, and the missing generic-500 envelope.

Domain: exec-standards

The execution engine has a reasonable staged-entry state machine (pre-submit staged_entries rows, comment-based idempotency probes, reconnect reconcile, kill-switch drain) but falls short of professional order-execution standards in the places that lose money. Symptom 1 (TP never attached, price runs to SL) has three provable mechanisms: (a) a correlated follow-up without a TP sends tp=0.0 in the SLTP modify, which ERASES any existing TP, and the orphan protective-TP watchdog is then permanently excluded by the sibling-rows filter in get_orphan_candidate_stage1s — critical; (b) there is zero post-trade verification/retry: a single failed or silently-skipped stage-1 align modify is terminal; (c) in multi-account mode the per-account signal_id fan-out plus an account-blind correlator mispairs follow-ups across accounts so the align lookup finds nothing. Beyond symptom 1: the connector blind-retries order-placing POSTs on ReadTimeout with no idempotency key (double-fill risk, plus fill-marked-failed untracked positions), the REST bridge runs blocking MT5 C calls on the event loop (causing the very timeouts that trigger retries and false heartbeat reconnects → the operator's known retcode-10027 cascade), retcode handling treats DONE_PARTIAL as total failure and has no retryable/terminal taxonomy with deviation hardcoded at 20 points, the close/kill-switch path hardcodes ORDER_FILLING_IOC that the open path itself documents brokers reject, and execute_signal never re-checks the kill switch mid-stagger so orders keep firing after an emergency flatten. Medium findings cover a structurally-always-false filled-limit detection, missing write-ahead intent + forward reconciliation on the unstaged path, an unguarded sl=None crash in the correlated path, and point-band stages that can never pass the zone-watch pre-flight. Symptoms 2 and 3 (CSRF dual-device, position-details 500) live in the dashboard/api domain, outside these files.

Domain: data-consistency

Data/state consistency and concurrency audit of the telebot executor pipeline (db.py, models.py, executor.py, trade_manager.py, bot.py, settings_store.py, api/actions.py, api/idempotency.py, plus signal_correlator.py, api/positions.py and api/stages.py to prove code paths). The architecture has no row locking or compare-and-set anywhere in the staged-entry state machine, and the per-account fan-out design (Executor builds a temp TradeManager per account, so per-account signal_ids and per-account orphan registrations flow through ONE shared correlator with independently shuffled account orders) directly produces the operator's "TP never attached" symptom: on a shuffle mismatch the follow-up pairs to the wrong account's orphan, the stage-1 SL/TP alignment silently no-ops, and the protective-TP watchdog is suppressed by an account-blind sibling check — the position rides its default SL with TP=0 to a full stop-out (critical). The awaiting_zone rows are double-fireable between the at-arrival fire loop and the 10s zone-watch poller (no claim, idempotency probe reads a pre-submit positions snapshot), and update_stage_status has no status precondition, so terminal states (including kill-switch cancellation) can be overwritten by in-flight fills. The dashboard's full-close writes pnl=0.0 into a status='closed' row that the history-sync loop (which only scans status='opened') can never repair, and limit-order trades stuck at status='pending' are similarly invisible — both make the dashboard permanently disagree with the broker. The drilldown 500 (symptom 3) is a proven asyncpg JSONB-as-string decode bug in get_position_drilldown (the executor defends against exactly this in two other places; db.py does not). Restart recovery is incomplete: the D-24 reconcile runs only on heartbeat reconnect, never at process startup, leaving phantom armed stages after a crash. The idempotency-key implementation (api/idempotency.py) itself is sound — insert-first claim, conflict detection, placeholder release — and no transactions are held across MT5 bridge calls; the pool (max 5) shows no proven exhaustion path. The CSRF multi-device symptom (2) lives in the auth layer, outside this domain.

A.2 Confirmed critical/high findings (verbatim, with verifier verdicts)

Findings were produced by independent domain auditors, so some overlap; the deduplicated view is in the main report.

A.2.1 Correlated follow-up bands never persist signal_sl/signal_tp — every zone-watch-fired stage opens with TP=0 and is never protected

Description. In `_handle_correlated_followup`, the staged_entries rows for bands 2..N omit the 'signal_sl' and 'signal_tp' keys, so `db.create_staged_entries` inserts NULL for both (`db.py:1043` uses `r.get()`). Bands that are in-zone at arrival fire immediately with the real TP (synth at `trade_manager.py:518-527` carries `target_tp`), but any band that ARMS and later fires via `executor._zone_watch_loop` -> `_fire_zone_stage` reads `stage.get('signal_tp')` -> None (`executor.py:1027`) and builds the synthetic signal with `target_tp=None` (`executor.py:1040`). `_execute_open_on_account` then sends the market order with `tp=jittered_tp` or `0.0 = 0.0` (`trade_manager.py:971`). The SL is also wrong: `resolved_sl` falls back to a default_sl_pips stop off the BAND edge (`executor.py:1009-1016, 1028`), not the signal's SL. Nothing ever attaches a TP afterwards: the orphan protective-TP watchdog only covers `stage_number=1` rows with NO siblings (`db.py:1130-1138` 'NOT EXISTS ... stage_number >= 2'), which these stages by definition fail. Result: a stage fills, runs with TP=0 to its (wrong) SL — exactly operator symptom 1. SECOND consequence of the same NULLs: the zone-watch price cascade (`executor.py:617-644`) falls back to `db.get_signal_targets(paired_signal_id)`, which is the ORPHAN's signals row logged with `sl=0/tp=0` (`trade_manager.py:269-276, db.py` log_signal defaults `sl=0.0/tp=0.0`), so line 644 'if not sig_dir or (sig_sl <= 0 and sig_tp <= 0): continue' skips the cascade — remaining stages keep firing even after price already hit the follow-up's real TP/SL. The EXEC2-01 comment in `executor.py:616-625` explicitly claims 'The stage row carries the follow-up's real SL/TP' — the correlated insert path never does.

Evidence. `trade_manager.py:488-503` rows dict keys: `signal_id, stage_number, account_name, symbol, direction, zone_low, zone_high, band_low, band_high, target_lot, snapshot_settings, mt5_comment, status` — no `signal_sl/signal_tp` (contrast `_handle_open` direct-zone rows at `trade_manager.py:710-711` which DO set them, and `_handle_text_only_open` at 329-330). `db.py:1043`: `'r.get("signal_sl"), r.get("signal_tp")' -> NULL`. `executor.py:1026-1028`: `'signal_sl = stage.get("signal_sl"); signal_tp = stage.get("signal_tp"); resolved_sl = sl_price if signal_sl is None else signal_sl'`. `executor.py:1040`: `'target_tp=signal_tp'`. `trade_manager.py:905` `'jittered_tp = signal.target_tp'` and `:971 'tp=jittered_tp or 0.0'`. `db.py:1134-1138` sibling NOT EXISTS excludes these stages from the protective watchdog.

FIX — Add 'signal_sl': `signal.sl` and 'signal_tp': `signal.target_tp` to the rows dict in `_handle_correlated_followup` (`trade_manager.py:488-503`), mirroring the direct-zone path at lines 710-711. Additionally, when the follow-up pairs, update the orphan's signals row (or the stage rows) with the real sl/tp so the `get_signal_targets` cascade fallback also works for pre-existing rows.

Adversarial verdict: CONFIRMED. Refutation attempt failed on all fronts. (1) Code exists as quoted: `trade_manager.py:488-503` rows dict for correlated follow-up bands omits

signal_sl/signal_tp, while the direct-zone path (710-711) and text-only path (329-330) both set them; db.py:1043 uses r.get() so NULL is inserted (comment even admits 'callers not yet updated ... insert NULL'). (2) Scenario reachable: bands not in-zone at arrival are armed (trade_manager.py:516-517) and fired later by executor._zone_watch_loop -> _fire_zone_stage, which reads the NULL fields (executor.py:1026-1028), substitutes a default_sl_pips SL off the band edge and target_tp=None (executor.py:1040); _execute_open_on_account then submits tp=jittered_tp or 0.0 = 0.0 (trade_manager.py:905, 971) — the D-08 backstop only guards sl<=0, and resolved_sl is a positive wrong price, so the order goes through. (3) No mitigation: the orphan protective-TP watchdog is hard-filtered to stage_number=1 rows with NO siblings >=2 (db.py:1130-1138), which correlated bands (stages 2..N with siblings) can never satisfy; the only other TP-modify paths are stage-1 align (stage 1 only) and an explicit MODIFY_TP channel signal. Second consequence also confirmed: cascade fallback get_signal_targets(paired_signal_id) hits the orphan's signals row logged with log_signal defaults sl=0.0/tp=0.0 (trade_manager.py:269-276, db.py:275-276), so executor.py:644 skips the cascade and later stages keep firing after the real TP/SL is hit (the stage-1 align does log real sl/tp but as a NEW signals row with a different id). The EXEC2-01 comment at executor.py:608-616 claiming 'The stage row carries the follow-up's real SL/TP' is false for this path — strong evidence of an intended-but-missed update. Matches operator symptom 1 exactly on the primary correlated-signal flow; loses money, so critical stands.

Verifier correction/refinement. None needed. Description, code citations, and both consequences verify exactly; the proposed fix (add signal_sl/signal_tp to the rows dict at trade_manager.py:488-503, plus updating the orphan's targets on pairing for the cascade fallback) is correct and mirrors the already-working direct-zone path.

A.2.2 Multi-account correlation pairs follow-ups to the WRONG account's signal_id — stage-1 TP alignment silently skipped, watchdog suppressed, false cascades

Description. Executor.execute_signal shuffles accounts (executor.py:108) and calls _execute_single_account per account (executor.py:136), each with a temp single-account TradeManager (executor.py:150-157). For an OPEN_TEXT_ONLY message this means db.log_signal runs ONCE PER ACCOUNT, creating a DISTINCT signal_id per account (trade_manager.py:269-276), and register_orphan is called once per account. The correlator keys orphans only by (symbol, direction) with no account dimension (signal_correlator.py:50, 69) and pair_followup pops the most-recent orphan LIFO (line 79). When the follow-up arrives, accounts are shuffled AGAIN independently, so with 2 accounts there is a ~50% chance (worse with more accounts) that account A pairs to the signal_id registered during account B's execution. Consequences, all provable: (1) trade_manager.py:407-409 get_stage_by_signal_account(paired_signal_id, acct_name, 1) returns None (that signal_id's stage-1 row belongs to the other account) -> the stage-1 SL/TP alignment is SILENTLY skipped (logger.debug only, trade_manager.py:473-477) -> the live position keeps default SL and TP=0. (2) The other account's bands were created under this account's signal_id, so db.get_orphan_candidate_stage1s' cross-account sibling check (db.py:1134-1138 has no account_name filter) sees stage_number>=2 rows and PERMANENTLY excludes the un-aligned stage-1 from the protective-TP watchdog. Net: position opened, TP never attached, runs to SL — symptom 1 on roughly every other multi-account text-only+follow-up sequence. (3) The D-16 cascade (executor.py:704-732) looks up 'telebot-{signal_id}-s1' via db.get_stage_by_comment (no account filter, db.py:1223-1230): account A has no such position but the row (account B's) is status='filled', so it wrongly cancels ALL of account A's armed stages for that signal_id. (4) The duplicate-direction bypass (trade_manager.py:829-835) uses sibling_prefix 'telebot-{paired_signal_id}-s', which does not match the account's actual stage-1 comment 'telebot-{own_signal_id}-s1', so band fills on that account are blocked with 'Already have a ... position open'.

Evidence. executor.py:107-108 'account_names = list(...); random.shuffle(account_names)'; executor.py:150-157 temp per-account TradeManager -> handle_signal; trade_manager.py:269-276 log_signal inside _handle_text_only_open (executed once per account); signal_correlator.py:69-79 'key = (symbol.upper(), direction.lower()); ... orphan = lst.pop()' — no account in the key; trade_manager.py:407-414 stage1 lookup by (paired_signal_id, acct_name, 1); trade_manager.py:473-477 silent skip; db.py:1134-1138 sibling NOT EXISTS with no account_name predicate.

FIX — Make signal identity account-independent for correlation: either log ONE signals row per message (executor should log before fan-out and pass signal_id down) so all accounts share the

signal_id, or key the correlator by (account_name, symbol, direction) and register/pair inside the per-account path. Also add account_name to the sibling NOT EXISTS in get_orphan_candidate_stage1s and to get_stage_by_comment-based cascade checks.

Adversarial verdict: CONFIRMED. Every cited line exists as quoted. executor.py:107-108 shuffles accounts per message and executor.py:150-157 gives each account a temp single-account TradeManager sharing one correlator (line 156). trade_manager.py:269-276 logs a signals row inside _handle_text_only_open, which runs once per per-account handle_signal call, so each account gets a DISTINCT signal_id and registers its own orphan. signal_correlator.py:69 keys orphans only by (symbol, direction) and line 79 pops LIFO; the follow-up's independent re-shuffle means correct pairing requires the follow-up order to be the exact reverse of the text-only order ($1/2$ for 2 accounts, $1/n!$ for n). All four consequences verified in code: (1) trade_manager.py:407-409 account-filtered lookup returns None for the wrong account's signal_id → silent debug skip at 473-477, leaving stage-1 with default SL and tp=0 (target_tp=None → 'jittered_tp or 0.0' at line 971); (2) db.py:1134-1138 sibling NOT EXISTS has no account_name predicate, so the other account's bands under this signal_id permanently exclude the un-aligned stage-1 from the protective-TP watchdog — and the failure is symmetric, so on a cross-pair BOTH accounts' stage-1 positions lose alignment AND watchdog protection; (3) executor.py:704-732 checks the stage-1 comment against this connector's positions only, then db.get_stage_by_comment (db.py:1223-1230, no account filter) returns the other account's filled row → cancel_unfilled_stages_for_signal wrongly cancels this account's armed bands; (4) trade_manager.py:829-835 sibling_prefix built from the wrong signal_id never matches the live stage-1 comment, blocking band fills as duplicates. No mitigating code (retries, dedup, account-aware correlation) exists. Only caveat found: the checked-in accounts.json has exactly one enabled account, and with one account the bug cannot fire — so it explains operator symptom 1 only if the deployed VPS config runs 2+ accounts (accounts.example.json shows the intended 2-account shape).

Verifier correction/refinement. Claim stands with three refinements. (a) Reachability caveat: the repo's accounts.json currently enables only ONE account ("Vantage Demo-10k"); with a single account pairing is always correct and none of this fires, so this defect explains the operator's live TP-never-attached symptom only if the deployed config runs 2+ accounts (the system's advertised multi-account design, per accounts.example.json). Severity is critical for any multi-account deployment; latent (medium) if production is provably single-account. (b) The impact is WORSE than described: cross-pairing is symmetric — when account A gets B's signal_id, B necessarily gets A's, so BOTH accounts' stage-1 positions are left with default SL and TP=0, and BOTH are excluded from the protective-TP watchdog (each signal_id has stage ≥ 2 sibling rows created by the other account). It is not "one account affected ~50% of the time" but "all accounts affected on every mispaired sequence". (c) Probability precision: with n accounts a fully correct pairing requires the follow-up shuffle to be the exact REVERSE of the text-only registration order — probability $1/n!$ ($1/2$ only for $n=2$); any deviation mispairs at least two accounts. Proposed fix is correct; logging one signals row before fan-out and passing signal_id down is the cleaner option since it also fixes the account-less

get_stage_by_comment/D-16 cascade and the sibling NOT EXISTS in
get_orphan_candidate_stage1s in one move (though adding account_name predicates to those
two queries is still advisable defense-in-depth).

A.2.3 Stage-1 TP attach is single-shot: one failed modify (REST timeout or broker invalid-stops 10016) leaves the position TP-less forever

Description. When a follow-up pairs to a text-only stage-1, the ONLY mechanism that attaches the TP to the already-open position is one `connector.modify_position` call (`trade_manager.py:424-426`). On failure it logs a warning, appends 'stage1_align_failed' to results, and moves on — there is no retry, no persisted retry state, and no later sweep. Failure is easy to hit: the REST server maps any modify retcode $\neq$ `TRADE_RETCODE_DONE` to HTTP 400 (`server.py:379-380` `_err("MODIFY_FAILED", f"retcode={result.retcode} ...")`), including 10016 `TRADE_RETCODE_INVALID_STOPS` (TP/SL within the broker's stops/freeze level of current price — likely when price has already run toward the TP) and 10025 `NO_CHANGES`; the connector turns that (or 3 consecutive timeouts, `mt5_connector.py:617-619`) into `OrderResult(success=False)`. Because the pairing already created sibling band rows, the orphan protective-TP watchdog is disqualified (`db.py:1134-1138` `NOT EXISTS stage_number>=2`), so unlike a true orphan (which retries every 10s until `position.tp != 0`, `executor.py:891-893`), an align-failed stage-1 never gets a second chance. The position keeps its 100-pip default SL and `TP=0` and rides to SL — symptom 1. Note the modify also bundles SL+TP in one `TRADE_ACTION_SLTP` request (`server.py:367-373`), so a TP that violates stops-level rejects the SL update too.

Evidence. `trade_manager.py:423-434` single try around `modify_position`; `:462-472` failure path only logs and records 'stage1_align_failed'; `server.py:379-380` 'if result.retcode $\neq$ `mt5.TRADE_RETCODE_DONE`: `_err(...)`'; `mt5_connector.py:601-607` error body `-> return None` - `> OrderResult(success=False)` at `trade_manager` via `connector.modify_position` (`mt5_connector.py:736-737`); `db.py:1134-1138` watchdog sibling exclusion.

FIX — Persist the desired SL/TP on the stage-1 row (`signal_sl/signal_tp`) at pairing time and add a retry sweep (the existing 10s zone-watch tick is a natural home): for any filled stage whose live position `tp==0` but whose row has `signal_tp` set, re-issue `modify_position`, mirroring the orphan watchdog's idempotency-by-live-TP pattern (`executor.py:890-893`). Split SL and TP into separate modifies or fall back to SL-only when the TP is rejected for stops-level.

Adversarial verdict: CONFIRMED. Every cited line verified verbatim. `trade_manager.py:423-426` is the sole TP-attach for a paired text-only stage-1; its failure path (`:462-472`) only logs and appends 'stage1_align_failed' — no retry, no persisted state. `server.py:379-380` maps any modify retcode $\neq$ `TRADE_RETCODE_DONE` (incl. 10016 `INVALID_STOPS`, likely exactly when price has run toward the TP) to HTTP 400 via `_err` (`server.py:59-67`), and `mt5_connector.py:601-607` returns `None` on any 400 body WITHOUT retry (only connect/read timeouts get 3 attempts, `:608-619`), yielding `OrderResult(success=False)` at `:736-740`. The bundled `TRADE_ACTION_SLTP` (`server.py:367-373`) means a rejected TP also discards the follow-up's SL. No rescue exists: pairing unconditionally creates sibling stage ≥ 2 rows right after

the align attempt (compute_bands stage_number=i+2, trade_manager.py:84; create_staged_entries at :506), which disqualifies the orphan protective-TP watchdog via db.py:1134-1138 NOT EXISTS sibling clause — unlike a true orphan, which retries every 10s until live tp != 0 (executor.py:889-893, cadence :533). The only other modify_position callers (_handle_modify_sl :1195, _handle_modify_tp :1236) require new Telegram signals; the zone-watch cascade only cancels unfilled rows. The fix premise also holds: stage-1 row is written with signal_tp=None (trade_manager.py:330) and never updated at pairing. Directly explains operator symptom 1 (TP never attached, position rides 100-pip default SL) and loses money when triggered, so critical per the rubric.

Verifier correction/refinement. Minor scope refinement to "TP-less forever": (a) if max_stages == 1 no sibling rows are created (compute_bands returns [] and the loop continues at trade_manager.py:484-486 before create_staged_entries), so the orphan watchdog would eventually attach a protective TP — the permanent-loss path requires the normal max_stages >= 2 configuration; (b) a later channel-issued modify-TP signal (trade_manager.py:1236) could repair it manually, but that is not an automatic code mitigation. Everything else in the finding, including the bundled SL+TP rejection side effect and the proposed fix, is accurate.

A.2.4 HTTP retry on ReadTimeout re-sends non-idempotent POST /api/v1/order — duplicate market orders

Description. `_request` retries up to 3 attempts on `httpx.ConnectError`, `ConnectTimeout` AND `ReadTimeout` for EVERY method/path, including `POST /api/v1/order`. A `ReadTimeout` means the request may have reached the server and `mt5.order_send` may have executed (the 15s client timeout can elapse while the single-threaded REST server — all MT5 calls run synchronously on the event loop via `_run`, `server.py:70-86` — is busy with another blocking call). The retry then submits a second identical market order, doubling exposure. The D-25 idempotency probe (`trade_manager.py:938-958`) runs BEFORE `open_order`, not between HTTP retries, so it cannot catch this. Conversely, if all 3 attempts time out, `open_order` returns `success=False` and the stage/trade is recorded as failed (`trade_manager.py:1044-1058`) while a real position may exist on the broker, untracked until reconnect reconciliation.

Evidence. `mt5_connector.py:586` 'for attempt in range(3)'; `:608-613` 'except (`httpx.ConnectError`, `httpx.ConnectTimeout`, `httpx.ReadTimeout`) ... continue' with no method/path discrimination; `open_order` posts via this path at `mt5_connector.py:704-713`.

FIX — Only retry idempotent GETs (and connection-level failures where the request provably never left, i.e. `ConnectError/ConnectTimeout`). For `POST /api/v1/order` on `ReadTimeout`, do a positions/orders probe by the unique `mt5_comment` before any resend, or make the server endpoint idempotent by rejecting a comment it has already executed.

Adversarial verdict: CONFIRMED. Every citation verifies. `mt5_connector.py:586-613`: `_request` retries up to 3 attempts on `httpx.ConnectError/ConnectTimeout/ReadTimeout` for ALL methods and paths, with only `asyncio.sleep(1)` between resends and no method/path discrimination; `open_order` posts `/api/v1/order` through it (`mt5_connector.py:704-713`); client timeout is 15s (line 577). Server side: `_run` (`server.py:70-86`) executes MT5 calls synchronously on the event loop, and the `/api/v1/order` handler (`server.py:253-355`) makes five sequential blocking MT5 calls and unconditionally calls `mt5.order_send` with no comment-based dedup — so a request whose response the client never read (`ReadTimeout` while the loop is stalled, or a slow `order_send`) still executes, and the retry executes it a second time (uvicorn processes the buffered request even after client disconnect; plain Starlette handlers are not cancelled). Mitigations refuted: (1) the D-25 idempotency probe (`trade_manager.py:938-958`) runs once before `connector.open_order`, never between HTTP retry attempts, and is staged-only — the unstaged path uses constant comment "telebot" with no probe at all; (2) D-24 reconnect reconciliation (`executor.py:256-277`) builds `by_comment={comment: position}`, so two duplicate positions with the same comment collapse to one dict entry and the duplicate is never detected, and it only runs on reconnect; (3) the all-attempts-timeout branch returns `success=False` (`mt5_connector.py:714-715`) and the trade is logged `status="failed"` (`trade_manager.py:1044-1058`) while a real broker position may exist, recoverable only at reconnect and only for staged rows — exactly as the claim states.

Severity critical is correct: a duplicated market order doubles live exposure (direct money loss). Caveat: the trigger requires an intermittent condition (server event-loop stall or broker-side `order_send` exceeding the 15s client timeout), so it is not every-trade frequent, but impact meets the critical bar. The proposed fix (retry only idempotent GETs/connection-level failures; probe positions by unique comment before resending `POST /order`, or server-side comment dedup) is sound, with one addition: the unstaged path must first get unique comments for any comment-based probe/dedup to work, since it currently sends the constant "telebot".

Verifier correction/refinement. Claim is accurate as written. Two additions: (a) the unstaged (non-staged) execution path uses the constant comment "telebot" (`trade_manager.py:938-939`), so it has no idempotency protection at all and the proposed comment-probe fix requires making unstaged comments unique first; (b) D-24 reconnect reconciliation (`executor.py:256-277`) cannot detect the duplicate even after the fact because its `by_comment` dict collapses same-comment positions to one entry.

A.2.5 CSRF cookie is session-lifetime while auth session persists 30 days, and the SPA never reseeds it — authenticated devices get permanent 403 'CSRF token invalid' (root cause of symptom 2)

Description. There is NO single global token, no token rotation that invalidates other devices, and no server-side session singleton — `verify_csrf_token` (api/deps.py:62-66) is a stateless per-request double-submit compare, and sessions are stateless Starlette signed cookies, so two devices logged in simultaneously are fully supported by the backend. The real mechanism: the `telebot_csrf` cookie is set WITHOUT `max_age/expires` in both `_issue_csrf` (api/auth.py:80-87) and `GET /auth/csrf` (api/auth.py:169-176), making it a non-persistent browser-session cookie, while the `telebot_session` auth cookie persists 30 days (dashboard.py:183 `max_age=30*24*60*60`, re-issued on every response by `SessionMiddleware`). Concrete failure: operator logs in on laptop and phone. Later one device's browser process restarts (laptop reboot) or the mobile OS evicts session cookies (iOS/Android do this aggressively on backgrounded browsers). That device still auto-authenticates — `App.tsx` (frontend/src/App.tsx:20-27) calls `GET /auth/me`, the persistent session cookie makes it 200, and `AppShell` renders immediately. But `document.cookie` no longer contains `telebot_csrf`. The SPA only seeds the CSRF cookie in the login view (frontend/src/auth/LoginView.tsx:41-45 `useEffect seedCsrf`; `grep` confirms `LoginView` is the ONLY `seedCsrf` call site), which never mounts for an authenticated user. Every trade-management mutation then sends an EMPTY `X-CSRF-Token` header (frontend/src/lib/http.ts:51-53, `readCookie` returns `""`), `verify_csrf_token` rejects it with 403 'CSRF token invalid' (api/deps.py:63-66), and the global error handler only recovers from 401, not 403 (frontend/src/lib/queryClient.ts:42-48) — so the device is stuck erroring on every action until a manual logout/re-login. The device whose browser kept running since its login retains the ephemeral cookie and keeps working — hence 'only one device can manage trades; the other gets csrf token invalid'. The same defect also breaks a SINGLE device after any browser restart within the 30-day session window.

Evidence. `api/auth.py:80-87` `resp.set_cookie(CSRF_COOKIE, token, httponly=False, samesite="lax", secure=secure, path="/")` — no `max_age` (same at :169-176). `dashboard.py:179-187` `app.add_middleware(SessionMiddleware, ... max_age=30 * 24 * 60 * 60`. `frontend/src/lib/http.ts:51-53` `if (STATE_CHANGING_METHODS.has(method)) { headers.set("X-CSRF-Token", readCookie("telebot_csrf")); }` with `readCookie` returning `""` when absent (`http.ts:34-40`). `api/deps.py:63-66` `if not cookie or not header or not _secrets.compare_digest(cookie, header): raise HTTPException(status_code=403, detail="CSRF token invalid")`. `frontend/src/App.tsx:20-27` renders `AppShell` on `/auth/me` success without any CSRF seeding. `frontend/src/auth/LoginView.tsx:41-45` is the only `seedCsrf()` call site (verified by `grep` over `frontend/src`). `frontend/src/lib/queryClient.ts:43` `if (error instanceof HttpError && error.status === 401)` — 403 has no handler.

FIX — Three coordinated changes: (1) Backend: add `max_age=30*24*60*60` (matching the session cookie) to both `set_cookie` calls in `api/auth.py` (`_issue_csrf` and the `/auth/csrf` handler) so the CSRF cookie survives browser restarts for the life of the session. (2) Frontend boot reseed: in `App.tsx` (or `AppShell`), after `/auth/me` succeeds, call `seedCsrf()` when `readCookie('telebot_csrf')` is empty. (3) Frontend self-heal: in `lib/http.ts`, on a 403 whose envelope message is 'CSRF token invalid', call `GET /api/v2/auth/csrf` once and retry the request a single time. Any one of these fixes the symptom; (1)+(2) together make it robust. No backend CSRF-check change is needed — the stateless double-submit already supports concurrent devices.

Adversarial verdict: CONFIRMED. Every cited code element verified verbatim: (1) `api/auth.py:80-87` and `:169-176` set the `telebot_csrf` cookie with no `max_age/expires`, making it a non-persistent browser-session cookie; (2) `dashboard.py:183` sets the `telebot_session` auth cookie `max_age=30*24*60*60` (30 days); (3) `grep` confirms `seedCsrf()` is called only from `LoginView.tsx:42`, and `router.tsx:30-37` + `App.tsx:20-27` confirm an authenticated user boots straight into `AppShell` without ever mounting `LoginView` or seeding CSRF; (4) `http.ts:51-53` sends `X-CSRF-Token` from `readCookie()`, which returns `""` when the cookie is absent (`http.ts:34-40`), and `api/deps.py:62-66` rejects empty header with 403 'CSRF token invalid'; (5) `queryClient.ts:43` only recovers from 401 — no 403 handler or `retry/reseed` anywhere. Failure is deterministically reachable: any browser restart or mobile cookie eviction within the 30-day session window leaves the device authenticated (persistent session cookie) but without the CSRF cookie, so every mutation 403s until manual logout/re-login. Cookies being per-device explains 'one device works, the other gets csrf token invalid'. No mitigating code exists. The claim's rejection of alternative mechanisms (global token, rotation, session singleton) is also correct — `verify_csrf_token` is a stateless double-submit compare and sessions are stateless signed cookies, so concurrent devices are backend-supported. Proposed fix is sound.

Verifier correction/refinement. Only a trivial citation nit: `seedCsrf()` is called at `frontend/src/auth/LoginView.tsx:42` (claim cited 41-45). Description and proposed fix are otherwise accurate. Severity high is appropriate: frequent operator-facing failure that blocks trade management on the affected device, but recoverable by re-login and not directly money-losing.

A.2.6 Position drilldown 500s on every staged trade: asyncpg returns JSONB snapshot_settings as str, get_position_drilldown calls .get() on it

Description. GET /api/v2/positions/{account}/{ticket} (api/positions.py:62, the SPA's PositionDrilldown query) calls db.get_position_drilldown(). For any trade that was opened via the staged-entry path, the staged_entries join (db.py:1274-1281, WHERE se.mt5_ticket = \$1) returns rows, and the loop does `settings = stage["snapshot\_settings"] or {}` then `settings.get("default\_sl\_pips")`. The pool is created with plain asyncpg.create_pool (db.py:60-65) and `set\_type\_codec` appears nowhere in the repo, so asyncpg returns the JSONB column as a Python str (it was written with json.dumps at db.py:1039). A non-empty JSON string is truthy, so `or {}` keeps the str, and `.get()` raises AttributeError: 'str' object has no attribute 'get'. api/errors.py registers handlers only for HTTPException and RequestValidationError (errors.py:83-88), so the AttributeError escapes as a bare, un-enveloped 500; the SPA's ErrorPanel then shows its generic failure text — the operator's "unable to get ...". Why it presents as active-trade-only: trade_manager.py:1002-1013 writes the same result.ticket into both staged_entries.mt5_ticket and trades.ticket, so the join hits for the entire lifetime of every staged trade, and the drilldown is only reachable from the open-positions view (closed trades go through /history, which never calls this endpoint). Text-only single-stage trades (no staged_entries row with that ticket) take the fallback branch at db.py:1296-1305 and do NOT crash — which is why only some trades' details 500. Proof that the runtime type really is str: the codebase itself defends against exactly this in two other consumers of the same column — executor.py:897-904 (`if isinstance(snapshot\_dict, str): snapshot\_dict = json.loads(snapshot\_dict)`) and dashboard.py:245-250 (`if isinstance(snapshot, str): snapshot = json.loads(snapshot)`). get_position_drilldown is the one reader without the guard.

Evidence. db.py:1284 `settings = stage["snapshot\_settings"] or {}` -> db.py:1291 `"sl\_at\_fill": settings.get("default\_sl\_pips"),` ; no codec: db.py:60-65 `\_pool = await asyncpg.create\_pool(dsn=database\_url, min\_size=2, max\_size=5, command\_timeout=30)` and repo-wide grep for set_type_codec returns nothing; write path db.py:1039 `r["target\_lot"], json.dumps(r["snapshot\_settings"])`; the guarded siblings: executor.py:898-900 `if isinstance(snapshot\_dict, str): ... json.loads(snapshot\_dict)` and dashboard.py:246-248 `if isinstance(snapshot, str): snapshot = json.loads(snapshot)`; join that triggers it: db.py:1278 `WHERE se.mt5\_ticket = \$1 AND se.account\_name = \$2`; ticket parity: trade_manager.py:1003 `await db.update\_stage\_status(stage\_row\_id, "filled", mt5\_ticket=result.ticket)` + trade_manager.py:1013 `ticket=result.ticket` in log_trade.

FIX — Best: register a JSONB codec once at pool creation — `asyncpg.create\_pool(..., init=lambda conn: conn.set\_type\_codec('jsonb', encoder=json.dumps, decoder=json.loads, schema='pg\_catalog'))` — then delete the scattered isinstance-str guards. Minimal: in get_position_drilldown, mirror executor.py:898-904: `settings = stage["snapshot_settings"]; if

`isinstance(settings, str): settings = json.loads(settings); if not isinstance(settings, dict): settings = {}`` before the `.get()`.

Adversarial verdict: CONFIRMED. Every element of the claim was verified against the actual code. (1) `db.py:1284` reads ``settings = stage["snapshot_settings"] or {}`` and `db.py:1291` calls ``settings.get("default_sl_pips")`` with no `str` guard. (2) The pool is created plain at `db.py:60-65` and a repo-wide `grep` for `set_type_codec` returns nothing, so `asynccpg`'s default JSONB decoding applies and returns `str`. (3) The column is JSONB NOT NULL (`db.py:229`) and is written with `json.dumps` as `$11::jsonb` (`db.py:1034,1039`), so every staged row yields a truthy `str` that survives ``or {}`` and then raises `AttributeError` on `.get()`. (4) The codebase itself proves the runtime type is `str`: `executor.py:898` and `:982` and `dashboard.py:246` all guard the same column with ``if isinstance(..., str): json.loads(...)`` — `get_position_drilldown` is the only reader without the guard. (5) Reachable: `api/positions.py:62` calls `db.get_position_drilldown` unguarded; `trade_manager.py:1003` writes `mt5_ticket=result.ticket` into `staged_entries` and `:1013` writes the same ticket into `trades`, so the join at `db.py:1278` matches for every filled staged trade and the crashing loop runs. Single-stage trades hit the fallback branch at `db.py:1296-1305` and don't crash, matching the intermittent symptom. (6) No mitigation: `api/errors.py:83-88` handles only `HTTPException/StarletteHTTPException/RequestValidationError`, so the `AttributeError` escapes as a bare 500; no tests reference `get_position_drilldown`. The finding stands as described, including the proposed fixes.

Verifier correction/refinement. None needed to the description or fix. Severity is high, not critical: the endpoint is read-only dashboard drilldown — it fails deterministically for every open staged trade (operator symptom 3) but does not lose money or block trade execution.

A.2.7 Correlated follow-up with no TP actively ERASES stage-1's TP, and the orphan watchdog is permanently excluded from re-attaching it

Description. In `_handle_correlated_followup`, when the follow-up signal carries no `target_tp`, `new_tp` is initialized to 0.0 and sent to `modify_position` anyway. On the REST server, `tp=0.0` is 'not None' so it is passed to `TRADE_ACTION_SLTP` verbatim — and in MT5, `tp=0` REMOVES the take-profit. So a follow-up that specifies only an entry zone + SL wipes whatever TP the stage-1 position had (including a watchdog-attached protective TP). Worse, the position can never be re-protected: `db.get_orphan_candidate_stage1s` explicitly excludes any stage-1 that has sibling rows ('AND NOT EXISTS (... sib.stage_number >= 2)'), and the follow-up just created those siblings. Result: position rides with SL only until price hits SL — this is a direct, provable mechanism for operator symptom 1 ('trade executes but TP is never attached, price runs to SL').

Evidence. `trade_manager.py:418-425`: ``new_tp = 0.0\n if signal.target_tp:\n new_tp = calculate_tp_with_jitter(...)\n ...\n modify_result = await connector.modify_position(stage1[\"mt5_ticket\"], sl=new_sl, tp=new_tp)``. `mt5-rest-server/server.py:372`: ``"tp": req.tp if req.tp is not None else pos.tp`` — 0.0 is not None, so 0.0 is written, clearing the TP. `db.py:1134-1138`: ``AND NOT EXISTS (SELECT 1 FROM staged_entries sib WHERE sib.signal_id = se.signal_id AND sib.stage_number >= 2)`` — once bands exist the watchdog never considers this position again.

FIX — When `signal.target_tp` is falsy, pass `tp=None` (preserve existing TP) instead of 0.0: ``new_tp = calculate_tp_with_jitter(...) if signal.target_tp else None``, and let the server's ``req.tp` is not None` guard keep `pos.tp`. Separately, extend the protective-TP watchdog (or a general reconciliation pass) to cover any bot-owned position with `tp==0` older than the correlation window, not only sibling-free stage-1s.

Adversarial verdict: CONFIRMED. Every cited line verified verbatim. `trade_manager.py:418-425` sends `tp=0.0` to `modify_position` whenever the correlated follow-up has no numeric TP (`signal_parser.py:113-120` provably yields `target_tp=None` for signals with no TP lines or only 'TP open', and `trade_manager.py:233-242` routes such OPENS into `_handle_correlated_followup` with no TP gate). `RestConnector` (`mt5_connector.py:729-735`) forwards 0.0 unchanged; `server.py:372` (``tp': req.tp if req.tp is not None else pos.tp`) treats 0.0 as an explicit value, so `TRADE_ACTION_SLTP` is sent with `tp=0`, which removes any existing TP in MT5. The handler then unconditionally inserts `stage>=2` sibling rows (`compute_bands stage_number=i+2`), and `db.py:1134-1138`'s NOT EXISTS sibling clause — used by the only protective-TP watchdog (`executor.py:824`) — permanently excludes the position from ever getting a protective TP. No other reconciliation re-attaches TPs. This is a proven mechanism for symptom 1 (position rides SL-only to stop-out). Minor narrative correction: the watchdog-attached-TP-erasure sequence is a timing race (correlator pairs only within the same window the watchdog waits out); the dominant reachable paths are (a) no-TP follow-up creating siblings that forever block the

watchdog from protecting a TP-less stage-1, and (b) wiping a TP set by a MODIFY_TP signal (trade_manager.py:1236) or manual terminal edit within the window. End state identical either way. Proposed fix (tp=None when target_tp falsy, plus broadening watchdog coverage) is correct.

Verifier correction/refinement. The "actively erases a watchdog-attached protective TP" sub-claim is only reachable via a timing race: the correlator pairs follow-ups only within correlation_window_seconds, while the protective-TP watchdog acts only after that window expires, so a watchdog TP normally does not exist yet when the follow-up modifies. The fully reachable failure paths are: (1) common case — text-only stage-1 has no TP, the no-TP follow-up writes tp=0.0 (no-op) but its sibling band rows permanently exclude the position from the protective-TP watchdog (db.py:1134-1138 NOT EXISTS), leaving it SL-only forever; (2) active erase — a TP set by a MODIFY_TP signal (trade_manager.py:1236) or a manual terminal edit inside the window is wiped by the tp=0.0 modify. Both produce the identical outcome (position runs to SL, symptom 1), so severity remains critical. Fix as proposed: new_tp = calculate_tp_with_jitter(...) if signal.target_tp else None, relying on server.py:372's is-not-None guard to preserve pos.tp; additionally extend watchdog/reconciliation coverage to bot-owned positions with tp==0 that have siblings.

A.2.8 REST connector retries POST /api/v1/order on ReadTimeout with no idempotency key — retried market orders can double-fill; fill-then-timeout leaves an untracked live position

Description. `_request` retries every request — including order-placing POSTs — up to 3 times on `httpx.ReadTimeout`. The REST server executes `mt5.order_send` synchronously on the event loop (see separate finding), so a slow broker or a queued request can exceed the 15s client timeout AFTER the order has actually filled. The retry then submits a second identical market order: double exposure with the same comment. If all 3 attempts time out, `_request` returns `None`, `open_order` reports `success=False`, and the bot marks the stage 'failed' / logs a failed trade — while a real position exists on the broker with no DB trade row managing it. The staged-comment reconcile (`executor._sync_positions`) only runs on reconnect, not after a timeout, and two duplicate positions share one comment so at most one is ever reconciled. This is the textbook gap that professional executors close with a client-generated idempotency key or unique client-order-id enforced server-side before `order_send`.

Evidence. `mt5_connector.py:586-619`: ``for attempt in range(3): ... except (httpx.ConnectError, httpx.ConnectTimeout, httpx.ReadTimeout) as exc: ... if attempt < 2: ... await asyncio.sleep(1) continue`` — no method/path filter, so POST `/api/v1/order` is retried. `server.py:253-355` has no dedup: nothing checks whether an order with `req.comment` already exists before `order_send`. The D-25 probe in `trade_manager.py:947-958` checks positions BEFORE the first submit, not between retry attempts inside `_request`.

FIX — Never blind-retry order-placing POSTs on `ReadTimeout`. Either (a) make `/api/v1/order` idempotent: server checks `positions_get/orders_get` for `req.comment` before `order_send` and returns the existing ticket; or (b) on client timeout, run a positions-by-comment probe (the D-25 logic already exists) before any resubmit, and adopt the position into the DB if found. Restrict the generic 3-attempt retry to GETs and idempotent DELETEs.

Adversarial verdict: CONFIRMED. Every cited code element exists as quoted. (1) `mt5_connector.py:586-613`: `_request` retries up to 3 times on `httpx.ReadTimeout` with no method/path filter, and `open_order` (line 704) posts `/api/v1/order` through it; client timeout is 15s (line 577). (2) `server.py:70-85`: `_run` executes MT5 calls synchronously on the event loop ('Call an MT5 function synchronously on the current (main) thread'), and `create_order` (253-355) chains 5 blocking calls, so fill-then-client-timeout is reachable under broker latency or request pileup (`trade_manager` polling shares the loop). (3) `server.py:253-355` has zero dedup — no `positions_get/orders_get` check on `req.comment` before `order_send`, no idempotency key in the schema. (4) The D-25 probe (`trade_manager.py:933-958`) runs once before `connector.open_order` and cannot see retries inside `_request`; it is also gated 'if staged:', so non-staged market orders (comment 'telebot') have no probe at all. (5) All-timeout orphan is permanent: `_request` returns `None` → `open_order` returns `success=False` (`mt5_connector.py:714-`

715) → trade logged status='failed' (trade_manager.py:1044-1058) and stage marked 'failed'; reconnect reconcile uses db.get_pending_stages which selects only status IN ('awaiting_followup','awaiting_zone') (db.py:1082), so a 'failed' stage is never adopted even after reconnect. (6) Duplicate fills share one comment and _sync_positions collapses positions into a by_comment dict (executor.py:256-260), so at most one duplicate could ever be reconciled. Could not refute on any axis. Severity: double-fill doubles realized loss on SL hit and the false 'failed' report invites manual re-entry (further exposure) — direct money loss, so critical per the rubric.

Verifier correction/refinement. Minor overstatement only: the orphaned/duplicated position is not fully unmanaged — SL and TP are embedded in the order request itself (server.py:298-299), so the broker bounds the loss at the SL. The core harm is doubled (or, after operator manual re-entry, tripled) exposure and a live position with no DB row, which trade_manager will never break-even/trail/close. Proposed fix is sound; note the reconcile gap is worse than claimed for non-staged trades: they all share the comment 'telebot' and have no reconcile path whatsoever.

A.2.9 No post-trade verification or retry for SL/TP attachment — a single failed/skipped stage-1 align is terminal, position rides unprotected

Description. Standard practice after `order_send` / `SLTP modify` is to positively confirm via a position query that SL/TP are actually set, and retry or alert if not. Here: (1) when the stage-1 align modify fails (requote, market-closed, invalid stops from jitter rounding, REST timeout), the failure is logged and appended to results — there is no retry queue, no periodic re-check, no operator alert path; (2) when the stage-1 row is not yet `status='filled'` or lacks `mt5_ticket` (e.g. the `update_stage_status` write raced or failed after a real fill), the align is skipped at `logger.debug` level — completely silent; (3) after `open_order` success nothing verifies the returned position actually carries the requested SL/TP. Any of these leaves the position with default SL and no TP, and since the follow-up created sibling rows, the orphan-TP watchdog will never touch it (db.py:1134-1138). Second concrete mechanism for symptom 1.

Evidence. `trade_manager.py:463-472`: ``else: logger.warning("%s: stage-1 align FAILED ticket=%s reason=%s", ...) results.append({... "status": "stage1_align_failed" ...})`` — nothing else happens. `trade_manager.py:473-477`: ``else: logger.debug("%s: skipping stage-1 align (stage1=%s)", ...)`` — silent skip when stage1 not filled/ticketless. `executor.py:955-959` (orphan watchdog modify failure): ``logger.warning("%s: orphan protective-TP FAILED ...")`` — watchdog does re-tick, but the align path has no equivalent loop.

FIX — Persist the desired SL/TP per position (the `signal_sl/signal_tp` columns already exist on `staged_entries`) and add a reconciliation pass in the 10s zone-watch tick: for every filled stage with a live position whose sl/tp deviate from the persisted targets, re-issue `modify_position` with bounded retries and notify the operator after N failures.

Adversarial verdict: CONFIRMED. Every cited line verifies. (1) `trade_manager.py:462-472`: stage-1 align failure only logs a warning and appends `{"status": "stage1_align_failed"}` — no retry, no re-check. (2) `trade_manager.py:473-477`: silent `logger.debug` skip when stage-1 is missing/not-filled/ticketless. (3) `db.py:1134-1138`: the orphan-TP watchdog's `NOT EXISTS(stage_number >= 2)` clause permanently excludes any stage-1 whose follow-up created sibling rows (created at `trade_manager.py:506`), so the watchdog can never repair a failed align. Mitigations checked and refuted: `mt5_connector.py:582-619` `_request` retries ONLY transport exceptions (`ConnectError/ConnectTimeout/ReadTimeout`); broker rejects (`ok:false` at 603-606 — requote, invalid stops, market closed) return `None` on the FIRST attempt with zero retries. `signal_sl/signal_tp` are only read at stage-fire time (`executor.py:617-618, 1026-1028`), never reconciled against live positions. No post-open or post-modify position query verifies SL/TP anywhere. `tests/test_trade_manager.py:489` (`test_followup_continues_band_fill_when_stage1_align_fails`) locks in the one-shot behavior as designed. The claim even understates the alert gap: the `stage1_align_failed` result reaches `notifier.notify_execution` (`bot.py:387-389`), but `notifier.py:39-69`'s status switch has no branch

for that status, so the failure is silently dropped from the Telegram execution report — the operator never sees it. The stage-1 text-only position opens with default SL and NO TP (trade_manager.py:330, 349-350), so a single failed/skipped align leaves it TP-less permanently — a direct, unmitigated mechanism for operator symptom 1 (trade executes, TP never attached, price runs to SL), which is realized money loss in production.

Verifier correction/refinement. Two refinements. (a) The failure-cause list should be split: broker rejects (requote/invalid-stops/market-closed) get ZERO retries (mt5_connector.py:603-606 returns None immediately on ok:false); only transport errors get 3 attempts (lines 608-613) before terminal failure. (b) The "no operator alert path" point is stronger than claimed: the stage1_align_failed result IS passed to notifier.notify_execution (bot.py:388-389) but notifier.py:39-69 has no case for that status, so it is silently omitted from the Telegram execution message. Proposed-fix caveat: the stage-1 row is inserted with signal_tp=None (trade_manager.py:330) and the align never persists the follow-up's SL/TP back onto the row, so the reconciliation pass must also write the desired SL/TP to the stage-1 row at follow-up time before comparing live positions against persisted targets.

A.2.10 REST bridge runs blocking MT5 C-extension calls directly on the asyncio event loop — serializes everything, causes the 15s timeouts, and false heartbeat failures that trigger re-login

Description. `_run` is 'async def' but simply calls the MT5 function inline: ``return fn(*args)``. `mt5.order_send`, `order_check`, `history_deals_get` can block for seconds (requotes, broker latency). While one blocks, EVERY other endpoint — `/ping`, `/price`, `/positions` — is frozen, because there is one event loop and no thread offload. Consequences chain: (a) concurrent bot requests (zone-watch `get_price` + `get_positions` every 10s, heartbeat ping every 30s, history sync) queue behind a slow `order_send` and hit the client's 15s timeout, feeding the double-fill retry defect; (b) a timed-out ping makes the heartbeat declare the connection dead and start `_reconnect_account`, which calls `mt5.login` — the exact trigger for the operator-confirmed retcode-10027 'AutoTrading disabled on account change' failure. Standard practice: dispatch MT5 calls to a dedicated single-thread executor so calls stay serialized (the MT5 lib is not thread-safe) but the event loop stays responsive.

Evidence. `server.py:70-86`: ``async def _run(fn, *args, **kwargs): ... if kwargs: return fn(*args, **kwargs) return fn(*args)`` — synchronous call in an async handler, no `run_in_executor`. `executor.py:176-180`: ``alive = await connector.ping(); if not alive: ... asyncio.create_task(self._reconnect_account(...))`` and `mt5_connector.py:627-642` `connect()` → `POST /connect` → `server.py:184` ``mt5.login``.

FIX — Create a module-level ``ThreadPoolExecutor(max_workers=1)`` and change `_run` to ``await asyncio.get_running_loop().run_in_executor(_mt5_executor, functools.partial(fn, *args, **kwargs))``. Single worker preserves MT5 call serialization while unblocking `/ping` and price reads.

Adversarial verdict: CONFIRMED. Confirmed with direct code evidence. `server.py:70-86` defines ``async def _run(fn, *args, **kwargs)`` that calls ``fn(*args)`` inline — no `run_in_executor`, no threadpool — and the docstring explicitly states it runs "synchronously on the current (main) thread". All endpoints are ``async def`` on a single-process uvicorn instance (`server.py:517-520`), so FastAPI gives them no threadpool offload: any blocking MT5 C-extension call (`order_send`, `order_check`, `history_deals_get`) freezes the entire event loop, including `/ping` and `/price`. The `/order` endpoint chains five blocking MT5 calls (`symbol_select`, `symbol_info_tick`, `symbol_info`, `order_check`, `order_send`; `server.py:264-329`). Client side confirmed: `mt5_connector.py:577` sets `timeout=15.0` and `:608` retries on `ReadTimeout`; concurrent load per server instance is real (heartbeat ping every 30s at `executor.py:165-176`, zone-watch `get_price/get_positions` every 10s at `executor.py:533/574/583`, history sync, orphan-TP checks). The reconnect chain is exactly as claimed: `executor.py:176-180` failed ping → `_reconnect_account` → `connector.connect()` (`mt5_connector.py:632`) → `POST /api/v1/connect` → `server.py:184` `mt5.login` — the operator-confirmed retcode-10027 trigger. No mitigation exists that refutes it; the only softening is that

_request retries 3x with 1s sleeps, so a false heartbeat failure requires ~45s of sustained loop blockage, not a single 15s stall. Shorter stalls still cause ReadTimeout retries (including retried POST /order, feeding the duplicate-fill risk).

Verifier correction/refinement. One overstatement: a single timed-out ping does NOT immediately trigger reconnect. `mt5_connector.py:586-613` retries each request 3 times (15s timeout each, 1s backoff), so `ping()` only returns False after ~45+ seconds of sustained event-loop blockage. The false-reconnect → `mt5.login` → `retcode-10027` path requires prolonged blocking (e.g. a broker-stalled `order_send` or a wide-window `history_deals_get` plus queued requests), not one slow call. Sub-45s stalls instead surface as ReadTimeout retries — which do include retried POST /order since the retry loop is not method-gated, feeding the double-fill defect. Fix caveat: the bare `mt5.last_error()` calls (`server.py:132, 153, 186, 314, 331, 419, 506`) must also be routed through the same single-thread executor, otherwise they run on a different thread than the MT5 calls after the fix.

A.2.11 Close path hardcodes ORDER_FILLING_IOC while the open path documents that this broker rejects IOC — kill-switch flatten and signal closes can fail with 'Unsupported filling mode'

Description. `create_order` carefully negotiates `type_filling` from `symbol_info.filling_mode`, with an in-code comment stating 'Market-execution brokers (e.g. Vantage) typically reject IOC'. Yet `close_position` hardcodes `ORDER_FILLING_IOC`. On a symbol/broker where the negotiated open mode was `FOK` or `RETURN` (supported-bitmask lacking IOC), every `close_position order_send` returns retcode 10030 (unsupported filling mode) → `_err("CLOSE_FAILED")` → the bot's `emergency_close` counts a failed_close and moves on. The kill switch — the last-resort loss limiter — cannot flatten positions on exactly the broker class the open-path comment says exists.

Evidence. `server.py:279-290` (open): ``sym_info = await _run(mt5.symbol_info, req.symbol); supported = int(sym_info.filling_mode) ...`` with fallback logic. `server.py:405-416` (close): ``request = { "action": mt5.TRADE_ACTION_DEAL, ... "type_filling": mt5.ORDER_FILLING_IOC, }`` — no negotiation. `executor.py:337-343`: failed closes are only counted/logged, never retried with a different mode.

FIX — Extract the filling-mode negotiation into a helper and use it in `close_position` (and add it to the deviation too). Optionally retry the close once with `RETURN` on retcode 10030.

Adversarial verdict: CONFIRMED. Verified verbatim: `server.py:415` hardcodes `"type_filling": mt5.ORDER_FILLING_IOC` in `close_position`, while `create_order` (`server.py:279-290`) negotiates from `symbol_info.filling_mode` with `FOK/RETURN` fallbacks — proving the codebase itself anticipates `DEAL` symbols lacking IOC support. On unsupported filling mode, `order_send` returns retcode 10030 != `TRADE_RETCODE_DONE`, so `close_position` raises `_err("CLOSE_FAILED")` (`server.py:423`) as HTTP 400 ok:false. No mitigation exists: `mt5_connector.py _request` (lines 586-613) retries only transport exceptions and maps ok:false to None (lines 603-606); connector `close_position` (742-756) has no filling-mode fallback (grep for 10030/filling: zero hits); `executor.py:337-343` `emergency_close` only increments failed_closes and logs; `trade_manager.py:1109/1149` close handlers only record status:failed. Refutation attempts failed — the defect stands.

Verifier correction/refinement. One overstatement: the open-path comment (`server.py:277-278`) says Vantage rejects IOC "for pending orders" — it does not establish that the operator's current broker lacks IOC on market DEALs. On symbols where the `DEAL` bitmask includes IOC, both open and close succeed, consistent with the operator not reporting failed closes today. The defect is latent: it fires with a guaranteed retcode 10030 on every bot-initiated close (kill switch, close/partial-close signals) exactly for symbols where the open path fell back to `FOK` or `RETURN` — positions that can be opened but never closed by the bot. Severity corrected from critical to high: guaranteed unmitigated kill-switch failure on a code-anticipated broker/symbol class, but

not proven to fire on the currently configured broker. Proposed fix (shared filling-mode negotiation helper reused in close_position, optional single retry with RETURN on 10030) is correct.

A.2.12 Retcode handling: `TRADE_RETCODE_DONE_PARTIAL` treated as total failure (partial fill becomes an untracked position); no retryable-vs-terminal retcode taxonomy; deviation hardcoded

Description. `create_order` treats any retcode `!= TRADE_RETCODE_DONE` as failure, returning `success=False` with `ticket=0`. `TRADE_RETCODE_DONE_PARTIAL` (10010) means volume WAS partially executed — the bot then logs a 'failed' trade (`trade_manager.py:1045-1057`) and marks the stage failed, while a live partial position sits on the broker unmanaged (no SL/TP verification, no DB row with its ticket). Likewise there is no retry for transient retcodes (10004 `REQUOTE`, 10021 `PRICE_OFF`, 10006 `REJECT`) versus terminal ones (10014 `INVALID_VOLUME`, 10016 `INVALID_STOPS`, 10027 `CLIENT_DISABLES_AT`): the connector only retries transport errors (`mt5_connector.py:608`), never business retcodes. Slippage tolerance is a hardcoded `"deviation": 20`` (points) with no per-symbol/config control — professional executors make `max-deviation` an explicit risk parameter and re-quote with fresh price on 10004.

Evidence. `server.py:340-347`: ``if result.retcodes != mt5.TRADE_RETCODE_DONE: return _ok({"success": False, "ticket": 0, ...})`` — 10010 falls in here despite ``result.volume`` being partially filled. `server.py:300`: `"deviation": 20,`` (also close path `server.py:412`). `trade_manager.py:1044-1058`: failure branch logs trade with ``ticket=0, status="failed"`` — the partial position's real ticket (`result.order`) is discarded server-side.

FIX — On 10010, return `success=True` with the actual filled volume and ticket so the bot records the real position. Classify retcodes: retry 10004/10021/10006 with fresh price and jittered backoff (bounded attempts); treat 10014/10016/10027 as terminal with distinct error codes so the bot can alert. Make deviation a config value passed per request.

Adversarial verdict: CONFIRMED. All cited code exists as quoted. `server.py:340-347` treats any retcode `!= TRADE_RETCODE_DONE` as total failure returning `ticket=0`, so `TRADE_RETCODE_DONE_PARTIAL` (10010) discards the real ticket/filled volume; 10010 is reachable because `server.py:282-283` selects `ORDER_FILLING_IOC` when the symbol supports it, and IOC permits partial fills. `server.py:300` and `:412` hardcode `"deviation": 20`. `mt5_connector.py:586-613` retries only httpx transport exceptions (`ConnectError/ConnectTimeout/ReadTimeout`); a business-retcode failure returns once with `success=False` and is never retried — grep across `executor.py/trade_manager.py/mt5_connector.py` for 10004/10010/10021/`REQUOTE/DONE_PARTIAL` finds zero hits, confirming no retryable-vs-terminal taxonomy. `trade_manager.py:1044-1058` logs the trade with `ticket=0 status="failed"`, and `executor.py:1081-1091` marks the stage terminally "failed", after which neither the D-25 idempotency probe (`trade_manager.py:933-958`, only runs before a resubmit) nor the reconnect reconcile (`executor.py:246-294`, only scans pending stages via `db.get_pending_stages`) will ever adopt the live partial position's ticket. However, the impact is overstated: the partial position is

NOT naked — the order request includes sl and tp (server.py:298-299), so the broker attaches them at fill; and management handlers (_handle_close trade_manager.py:1107, _handle_close_partial :1142, SL/TP-move paths :1179/:1225) operate via connector.get_positions(symbol) symbol-wide scans rather than DB tickets, so follow-up channel signals (close, BE, TP move) still reach the orphan position. Actual damage is DB/accounting: missing trades row (update_trade_close writes to a nonexistent row, PNL/history gap), stage marked failed frees lot budget while volume actually filled (slight over-exposure), and dashboard invisibility. Combined with the rarity of partial fills at retail lot sizes, this is an edge-case correctness risk plus hygiene, not a critical money-loser.

Verifier correction/refinement. The partial-fill position is not "unmanaged with no SL/TP": server.py:298-299 puts the requested sl/tp in the order, so the broker attaches them to the partially filled position, and trade_manager's close/modify handlers (lines 1107, 1142, 1179, 1225) scan live MT5 positions by symbol rather than DB tickets, so subsequent management signals still act on it. The real consequences are: no trades DB row for the live ticket (PNL/history-sync writes miss), stage/risk accounting treats the filled volume as failed (potential over-exposure when lot budget is reused), and the position is invisible in the dashboard. The no-retry-taxonomy and hardcoded deviation=20 observations are accurate as stated. Proposed fix stands (return success=True with filled volume/ticket on 10010; classify transient vs terminal retcodes; make deviation configurable).

A.2.13 Kill-switch race: execute_signal checks the pause flag only once at entry — an in-flight staggered signal keeps opening positions after emergency_close has flattened the account

Description. execute_signal reads _trading_paused once, then iterates accounts with deliberate random 1-5s stagger delays between each (total up to $\sim(N-1)*5s$). If the operator hits the kill switch mid-iteration, emergency_close sets _trading_paused, drains staged_entries, and closes all positions — but the in-flight execute_signal continues to the next account and submits new market orders, reopening exposure seconds after the flatten. The zone-watch loop got this right (mid-tick re-checks at executor.py:683-684 and 760-761: 'D-21 re-check BEFORE submit'); the primary signal path did not.

Evidence. executor.py:104-105: `if self.\_trading\_paused: return [...]` — only at function entry. executor.py:112-119: `for i, acct\_name in enumerate(account\_names): if i > 0: delay = random.uniform(self.cfg.stagger\_delay\_min, self.cfg.stagger\_delay\_max) ... await asyncio.sleep(delay)` — no re-check of _trading_paused inside the loop before `await self.\_execute\_single\_account(...)` at line 136.

FIX — Re-check `self.\_trading\_paused` at the top of each loop iteration (after the sleep, before _execute_single_account) and break with a 'skipped: kill switch' result — mirroring the zone-watch D-21 mid-tick guard.

Adversarial verdict: CONFIRMED. Every element of the claim checks out against the code. (1) executor.py:104-105 is the sole _trading_paused check in execute_signal, at function entry only. (2) The account loop (executor.py:112-137) sleeps random.uniform(stagger_delay_min, stagger_delay_max) — defaults 1.0/5.0s per models.py:135-136 — between accounts and calls _execute_single_account (line 136) with no pause re-check. (3) No downstream guard exists: _execute_single_account delegates to TradeManager.handle_signal, and trade_manager.py contains zero references to the pause flag. (4) The race is reachable: bot.py:411 init_dashboard(executor, ...) runs uvicorn in the same process/event loop as the Telethon handler, so api/actions.py:248 `await executor.emergency\_close()` can interleave with an in-flight `await executor.execute\_signal(...)` (bot.py:387) suspended at the stagger sleep or an MT5 REST await; emergency_close sets _trading_paused (executor.py:311), drains stages (318), and flattens all accounts (329-345), after which the resumed execute_signal submits new market orders for remaining accounts. (5) The claimed asymmetry is real: the zone-watch loop re-checks _trading_paused mid-tick at executor.py:683-684 and again before submit at 759-761 (explicit 'D-21 re-check BEFORE submit' comments), proving the authors consider this guard necessary but omitted it from the primary signal path. The proposed fix (re-check after each stagger sleep, before _execute_single_account) is correct and mirrors the existing D-21 pattern. Severity note: the window is a few seconds per account (only when kill switch fires inside an in-flight signal's stagger window), and reopened positions still carry SL/TP — so it defeats the kill-

switch guarantee at the worst possible moment (high), but is not a routine loss path or trading blocker (not critical).

Verifier correction/refinement. Description is accurate with one nuance: `emergency_close` does not get "reopened" positions from the flattened exposure itself — the in-flight `execute_signal` opens the NEW signal's positions on accounts not yet processed when the kill switch fired. These carry normal SL/TP. Severity is high rather than critical: it requires the kill switch to land inside an in-flight signal's stagger window (seconds), but when it does, the operator is left with live exposure while believing the account is flat. Proposed fix is correct; place the re-check after the stagger sleep and before the disabled/reconnecting checks at `executor.py:121-134`, breaking with a 'skipped: kill switch' result, mirroring the D-21 guards at `executor.py:683-684` and `759-761`.

A.2.14 Multi-account signal identity is broken: per-account temp TradeManager mints a different signal_id per account, and the correlator pairs orphans by (symbol,direction) only — cross-account mispairing; the alternative single-TM path collides on UNIQUE(mt5_comment)

Description. Executor._execute_single_account wraps EACH account in a temp TradeManager and calls handle_signal per account. _handle_text_only_open then calls db.log_signal at the top — so one Telegram text-only signal produces N distinct signal_ids (one per account), each registered as a separate orphan in the SHARED correlator, keyed only by (symbol, direction) with no account. When the follow-up arrives (also executed per-account), each account's pair_followup pops the MOST RECENT orphan regardless of which account created it: account A's follow-up can pair to account B's signal_id. Then get_stage_by_signal_account(B_id, 'A', 1) finds nothing → stage-1 align silently skipped → A's stage-1 keeps default SL and no TP (feeds symptom 1); A's band rows are persisted under B's signal_id, corrupting the D-16 stage-1 cascade (it probes comment 'telebot-{B_id}-s1' against A's positions). The design can't be fixed by reverting to one shared TM either: with multiple connectors in one handle_signal call, all accounts share one signal_id, so stage rows collide on the UNIQUE mt5_comment ('telebot-{id}-s1') and every account after the first is skipped as 'duplicate' — a collision db.py itself documents.

Evidence. executor.py:150-157: `temp\_tm = TradeManager(connectors={target\_account: ...}, ...); temp\_tm.correlator = getattr(self.tm, "correlator", None); return await temp\_tm.handle\_signal(...)` trade_manager.py:269-284: `signal\_id = await db.log\_signal(...)` then `await correlator.register\_orphan(signal\_id=signal\_id, symbol=..., direction=...)` — runs once per account via the temp TM. signal_correlator.py:69-79: `key = (symbol.upper(), direction.lower()); ... orphan = lst.pop()` — no account dimension. db.py:1239-1241: 'mt5_comment UNIQUE makes get_stage_by_comment() collide across accounts'. trade_manager.py:336-339: `except Exception as exc: # UNIQUE(mt5\_comment) violation ... "reason": "duplicate"`.

FIX — Give a Telegram message ONE canonical signal_id: log the signal once in Executor.execute_signal (or key by message id) and pass signal_id into the per-account execution; make the correlator key (account, symbol, direction) — pair_followup already runs in a per-account temp TM, so pass the account name through; include account_name in the mt5_comment (or drop the global UNIQUE in favor of UNIQUE(mt5_comment, account_name), which the schema comment suggests is already the operational reality).

Adversarial verdict: CONFIRMED. Every cited line verifies: executor.py:150-157 creates a per-account temp TradeManager sharing one correlator; trade_manager.py:269-284 logs a new signal_id and registers an orphan once per account for one Telegram message; signal_correlator.py:69-79 keys orphans by (symbol, direction) only and pops the most recent,

so with ≥ 2 accounts a follow-up pairs to another account's `signal_id` (independent shuffles make this ~50% likely with 2 accounts). On mispair, `get_stage_by_signal_account` (`trade_manager.py:407`) returns `None` and the stage-1 SL/TP align is silently skipped (debug-only log at 473-477) — stage-1 keeps default SL and TP=0 (symptom 1). Worse, the mispaired account's band rows persist under the other account's `signal_id`, and the D-16 cascade (`executor.py:704-732`) probes 'telebot-{id}-s1' against the wrong account's positions, misses, finds the other account's filled DB row, and spuriously cancels the armed bands. The claimed single-shared-TM alternative collision is documented in the code itself (`db.py:230` `UNIQUE(mt5_comment);` `db.py:1239-1240` comment; `trade_manager.py:337-339` 'duplicate' skip). No mitigation exists: no message-id dedup, no account dimension in the correlator, no multi-account correlation test. Caveat: the checked-in `accounts.json` has exactly one enabled account, under which the mispair is unreachable — but `accounts.example.json` ships two enabled accounts and the executor exists specifically for multi-account staggering, so the defect is certain in the intended deployment mode. If production currently runs one account, symptom 1 has a different cause and this bug is latent.

Verifier correction/refinement. Finding stands as described. Two refinements: (1) the defect only manifests with ≥ 2 enabled accounts — the repo's local `accounts.json` has one enabled account, so if production mirrors it, this cannot be the cause of the operator's currently observed symptom 1 (that needs a separate explanation on single-account); `accounts.example.json` ships two enabled accounts, making this critical for the advertised multi-account deployment. (2) The D-16 corruption's concrete effect is spurious cancellation: the mispaired account's armed bands are cancelled via `cancel_unfilled_stages_for_signal` because stage-1's comment is probed against the wrong account's MT5 positions — in addition to the missed stage-1 SL/TP alignment. The proposed fix (one canonical `signal_id` per Telegram message + account-keyed correlator + per-account `mt5_comment` uniqueness) is consistent with the code, and `db.py:1236-1240` already acknowledges the cross-account comment collision.

A.2.15 Cross-account orphan/follow-up mispairing silently skips stage-1 TP/SL alignment — TP never attached, trade rides to SL

Description. With ≥ 2 accounts, an OPEN_TEXT_ONLY signal is executed once per account via a temp TradeManager, so `_handle_text_only_open` logs a SEPARATE signal_id per account and registers a SEPARATE orphan per account in the shared correlator. The follow-up OPEN is also executed per account, and each per-account call pops the MOST-RECENT orphan from the shared correlator. Because `executor.execute_signal` calls `random.shuffle(account_names)` independently for the text-only and the follow-up signal, the pop order matches the registration order only by luck (1/2 for 2 accounts, worse for more). On a mispair, account A pairs with account B's signal_id; the stage-1 alignment lookup `get_stage_by_signal_account(paired_signal_id, acct_name, 1)` filters by account_name and returns None (B's stage-1 row belongs to B), so the code takes the silent 'skipping stage-1 align' branch — the already-open stage-1 market position NEVER receives the follow-up's SL/TP. Worse, the EXEC2-05 orphan protective-TP watchdog is also suppressed: the mispaired follow-up inserts sibling stage rows (`stage_number ≥ 2`) under that signal_id, and `get_orphan_candidate_stage1s` excludes any stage-1 whose signal_id has siblings (the NOT EXISTS subquery has no account_name filter, `db.py:1134-1138`). Result: both accounts' stage-1 positions keep only the default SL and TP=0 forever — exactly operator symptom (1): trade executes, TP never attached, price runs to SL. Additionally the mispaired bands can never fire: the duplicate-direction check uses sibling_prefix `f"telebot-{signal_id}-s"` (`trade_manager.py:829-835`) which doesn't match the account's real stage-1 comment, so every band submit is skipped.

Evidence. `executor.py:107-108` `account_names = list(self.tm.connectors.keys()); random.shuffle(account_names)` (independent shuffle per signal); `executor.py:150-157` temp TM per account with `temp_tm.correlator = getattr(self.tm, "correlator", None)` (shared); `trade_manager.py:269-276` `signal_id = await db.log_signal(...)` inside `_handle_text_only_open` — one per per-account invocation; `trade_manager.py:280-284` `await correlator.register_orphan(signal_id=signal_id, ...)` per account; `signal_correlator.py:79` `orphan = lst.pop()` (most-recent wins, one-to-one); `trade_manager.py:407-409` `stage1 = await db.get_stage_by_signal_account(paired_signal_id, acct_name, 1)` and `473-477` `else: logger.debug("%s: skipping stage-1 align (stage1=%s)", ...)` — silent skip; `db.py:1134-1138` `AND NOT EXISTS (SELECT 1 FROM staged_entries sib WHERE sib.signal_id = se.signal_id AND sib.stage_number >= 2)` — no account_name filter, watchdog suppressed for the wrong account's orphan.

FIX — Make correlation account-aware: register/pair orphans keyed by (account_name, symbol, direction) — i.e., call `correlator.register_orphan/pair_followup` with the account name from inside the per-account TradeManager, or move pairing up to `Executor.execute_signal` so ONE pairing decision is made per follow-up signal and the same paired signal_id map {account ->

orphan_signal_id} is passed to every account's handler. Also add account_name to the sibling NOT EXISTS clause in get_orphan_candidate_stage1s, and log the 'skipping stage-1 align' branch at WARNING (it is a money-losing silent path).

Adversarial verdict: CONFIRMED. Every cited line exists as quoted and the failure path is mechanically sound. (1) executor.py:107-108 re-shuffles account order independently for the text-only and follow-up signals; each account runs through a temp TradeManager sharing ONE correlator (executor.py:150-157). (2) _handle_text_only_open (trade_manager.py:269-284) logs a separate signal_id and registers a separate orphan per account, but the correlator key is only (symbol.upper(), direction.lower()) (signal_correlator.py:50) — account is not recorded. (3) pair_followup pops LIFO (signal_correlator.py:79), so correct pairing requires the follow-up shuffle to be the exact reverse of the registration shuffle — 1/2 mispair probability with 2 accounts, worse with more. (4) On mispair, get_stage_by_signal_account filters WHERE signal_id=\$1 AND account_name=\$2 (db.py:1242-1247), returns None, and the code takes the DEBUG-level 'skipping stage-1 align' branch (trade_manager.py:473-477) — the open stage-1 position keeps its default SL and TP=0 on BOTH accounts. (5) The mispaired follow-up inserts stage>=2 sibling rows (compute_bands emits stages 2..N, trade_manager.py:83-86) under the wrong signal_id, and get_orphan_candidate_stage1s' NOT EXISTS (db.py:1134-1138) has neither an account_name filter NOR a status filter, so the EXEC2-05 protective-TP watchdog is suppressed permanently — even after cancellation. (6) The band duplicate-direction bypass uses sibling_prefix f"telebot-{signal_id}-s" (trade_manager.py:829-835) which doesn't match the account's real stage-1 comment, so at-arrival band fires are skipped. I also found an amplifier the auditor missed: the D-16 cascade in executor.py:704-732 looks up the mispaired stage-1 comment via get_stage_by_comment (no account filter), sees it 'filled' in DB but absent from this account's positions, and cancels all mispaired bands within ~10s — cementing the outcome. No test covers multi-account correlation (grep of tests/ finds none). REACHABILITY CAVEAT: the bug requires >=2 enabled accounts. The local gitignored accounts.json has exactly ONE enabled account (Vantage Demo-10k), under which mispairing is impossible — so this cannot be proven to be the cause of the operator's currently observed symptom (1) unless the production VPS runs >=2 accounts (accounts.example.json ships two: FN-6k, FN-10k; the architecture is explicitly multi-account). The defect stands; its link to today's symptom is unproven from this repo.

Verifier correction/refinement. The defect and mechanism are exactly as described, with two refinements. (a) Severity: 'critical' assumes >=2 enabled accounts; the checked-in local accounts.json has only one, so the path is latent in the visible deployment and cannot explain the operator's current symptom (1) unless production runs multi-account. If production does run >=2 accounts, critical is correct (50% chance per correlated sequence that BOTH accounts ride default-SL/TP=0). (b) Two additions to the description/fix: the D-16 cascade (executor.py:704-732, via get_stage_by_comment which also lacks an account filter) actively cancels the mispaired bands within one 10s tick — and because the NOT EXISTS sibling check in get_orphan_candidate_stage1s (db.py:1134-1138) has no STATUS filter either, cancelled siblings still suppress the protective-TP watchdog forever. The fix should add both an account_name

filter and a status filter (or restrict to non-terminal statuses) to that subquery, in addition to the proposed account-aware correlation and WARNING-level logging of the skip branch.

A.2.16 Duplicate stage fire: at-arrival band firing races the 10s zone-watch loop with no row claim/lock — double broker exposure with the same comment

Description. Both `_handle_correlated_followup` and the direct-zone `_handle_open` insert `staged_entries` rows with `status='awaiting_zone'` and THEN fire in-zone bands in the same coroutine. The row is only flipped to 'filled' AFTER `connector.open_order` returns (`trade_manager.py:1002-1003`), and the fire path performs multiple awaited network calls first (`get_positions`, `get_price`, daily-stat reads, the order itself). The independent `_zone_watch_loop` task polls ``SELECT ... WHERE status='awaiting_zone'`` every 10s (`executor.py:533`, `db.py:1097-1105`) with no `SELECT ... FOR UPDATE`, no 'firing' claim status, and no compare-and-set. If a tick lands inside the multi-second fire window, both paths run `_execute_open_on_account` for the same stage. The D-25 idempotency probe (`trade_manager.py:940-958`) cannot save this: it checks the DB row status (still 'awaiting_zone') and a positions snapshot fetched BEFORE the first order has landed on the broker, so both submits pass the probe and two market orders are opened with the identical comment. Both then call `update_stage_status(..., 'filled')` which has no WHERE status guard (`db.py:1055-1062`) — last writer wins, one broker position is untracked in `staged_entries`. Result: 2x intended lot exposure and dashboard/stages state that disagrees with the broker.

Evidence. `trade_manager.py:715` ``stage_ids = await db.create_staged_entries(rows)`` followed by 720-749 at-arrival fire loop (same pattern at 506-540 for correlated path); `trade_manager.py:1002-1003` ``if stage_row_id is not None: await db.update_stage_status(stage_row_id, "filled", ...)`` — only after order success; `executor.py:549` ``rows = await db.get_active_stages()`` in a loop with ``await asyncio.sleep(10)``; `db.py:1097-1105` plain SELECT, no locking; `db.py:1055-1069` `update_stage_status` has no status precondition; `trade_manager.py:947-958` probe iterates ``positions`` fetched at line 809, before order submission.

FIX — Claim the row atomically before submitting: ``UPDATE staged_entries SET status='firing' WHERE id=$1 AND status='awaiting_zone' RETURNING id`` and only submit when the claim succeeds (both in the at-arrival fire loops and in `_zone_watch_loop/_fire_zone_stage`). Add a status precondition to `update_stage_status` (e.g. ``WHERE id=$1 AND status IN ('awaiting_zone','firing')`` for the 'filled' transition) so terminal states can never be overwritten.

Adversarial verdict: CONFIRMED. Every citation checks out verbatim: rows are inserted with `status='awaiting_zone'` (`trade_manager.py:707,715`; correlated path 502,506) and only flipped to 'filled' after `connector.open_order` returns (`trade_manager.py:1002-1003`; `executor.py:1069-1072`). The zone-watch loop polls a plain SELECT WHERE `status='awaiting_zone'` every 10s (`executor.py:533,549`; `db.py:1097-1105`) with no FOR UPDATE, no claim status, and `_fire_zone_stage` (`executor.py:961-1050`) submits without any pre-submit status flip. `grep` across `bot.py/executor.py/trade_manager.py/db.py` finds zero `asyncio.Lock/Semaphore` — both

paths interleave freely on the same event loop (zone-watch task started at executor.py:71). The guards do not mitigate: the duplicate-direction check explicitly allows same-signal comments (trade_manager.py:829-833 sibling_prefix continue), and the D-25 probe (940-958) reads a DB status that is still 'awaiting_zone' plus a positions list fetched at line 809 — if the first order hasn't landed on the broker by then, both submits pass and two market orders open with the identical comment. update_stage_status (db.py:1055-1069) has no WHERE-status guard, so both writers stamp 'filled' and one broker position is left untracked in staged_entries. The proposed atomic-claim fix is correct. Minor correction only: the race window is not the entire multi-second fire path — the second caller re-fetches positions at trade_manager.py:809, so the window is from that fetch until the first order lands on the broker (order in-flight REST latency plus the awaits between lines 809 and 960). Still hundreds of ms to seconds against a 10s poll, so realistically hit whenever a signal arrives with price already inside a band.

Verifier correction/refinement. Narrow the failure-window description: the D-25 probe inside the second _execute_open_on_account call uses positions re-fetched at trade_manager.py:809 (not only the zone-watch tick's earlier snapshot at executor.py:583), so the double fire requires the second call's line-809 get_positions to run before the first order lands on the broker. This is the first order's in-flight window (REST bridge + mt5.order_send, typically 0.2-2s) plus the second path's pre-submit awaits — narrower than 'the whole multi-second fire path' but still readily hit by the 10s poll whenever a stage fires at-arrival. Everything else in the claim, including the proposed row-claim + status-precondition fix, is accurate.

A.2.17 Position drilldown 500: asyncpg returns snapshot_settings JSONB as str, get_position_drilldown calls .get() on it

Description. `init_db` creates the pool with no `set_type_codec`, so `asyncpg` returns JSONB columns as raw JSON strings (the codebase itself proves this: `executor.py` defensively `json.loads` `snapshot_settings` in two places). `get_position_drilldown` does ``settings = stage["snapshot_settings"] or {}`` — a non-empty JSON string is truthy, so ``settings`` is a str — then calls ``settings.get("default_sl_pips")``, raising `AttributeError: 'str' object has no attribute 'get'` → unhandled 500 from GET `/positions/{account}/{ticket}` (`api/positions.py:62`). Since the EXEC2-06 direct-zone path now writes `staged_entries` rows for EVERY zone OPEN (`trade_manager.py:693-715`) and the fill populates `mt5_ticket`, essentially every live position matches the ``WHERE se.mt5_ticket=$1`` join and hits this branch. This is operator symptom (3): opening an active trade's details returns a 500 'unable to get ...'.

Evidence. `db.py:1284-1291` ``settings = stage["snapshot_settings"] or {}`` ... ``"sl_at_fill": settings.get("default_sl_pips")``; `db.py:57-67` `init_db` has no type codec (grep for `set_type_codec: none`); proof `asyncpg` hands back str in this codebase: `executor.py:898-903` ``if isinstance(snapshot_dict, str): snapshot_dict = json.loads(snapshot_dict)`` and identical guard at `executor.py:981-987`; `api/positions.py:62` ``detail = await db.get_position_drilldown(ticket, account)`` with no exception handling.

FIX — Either register a JSON codec at pool init (``await conn.set_type_codec('jsonb', encoder=json.dumps, decoder=json.loads, schema='pg_catalog')`` via `create_pool(init=...)`), or mirror the executor's guard in `get_position_drilldown`: ``settings = stage["snapshot_settings"];` `settings = json.loads(settings) if isinstance(settings, str) else (settings or {})``.

Adversarial verdict: CONFIRMED. Every element of the claim checks out against the code. (1) `db.py:57-67` `init_db` creates the `asyncpg` pool with no `init=` callback and no `set_type_codec` anywhere in the repo, so `jsonb` columns are returned as raw JSON strings. (2) `snapshot_settings` is JSONB NOT NULL (`db.py:229`) and is always written via `json.dumps` (`db.py:1039`), so on read it is a truthy str (even `'{}'`), meaning the ``or {}`` at `db.py:1284` never converts it; `settings.get('default_sl_pips')` at `db.py:1291` raises `AttributeError: 'str' object has no attribute 'get'`. (3) The codebase itself proves the str behavior: `executor.py:897-904` and `981-988` both defensively `json.loads` `snapshot_settings` when `isinstance(str)` — same column, same pool. (4) Reachable in the normal flow: `trade_manager.py:693-715` writes `staged_entries` rows for every direct-zone OPEN, fills `populate mt5_ticket` via `update_stage_status` (`db.py:1052-1061`, callers at `trade_manager.py:357/534`, `executor.py:271/1057`), so the drilldown join `WHERE se.mt5_ticket=$1` (`db.py:1278`) matches live staged positions and enters the crashing loop. (5) `api/positions.py:62` has no exception handling — only `None` maps to 404 — so the `AttributeError` surfaces as an unhandled 500, matching operator symptom (3). No test references `get_position_drilldown`; no mitigation exists. Minor framing correction: trades with no

staged_entries row take the db.py:1296 fallback and do not crash, so it is not literally every position — but all staged/zone positions (the normal case under EXEC2-06) do 500. Both proposed fixes are valid.

Verifier correction/refinement. Scope nuance only: positions with no staged_entries row (pure single-shot trades) take the fallback branch at db.py:1296 and render correctly, so the 500 affects staged/zone positions specifically — which is the dominant case since trade_manager.py:693-715 writes staged rows for every zone OPEN. Severity is high (blocks the operator's drilldown for active trades, a frequent operator-facing failure) rather than critical, since it does not itself lose money or block trade execution.

A.2.18 Dashboard full-close permanently records `pnl=0.0` — analytics/history disagree with broker

Description. `POST /positions/{account}/{ticket}/close` calls `db.update_trade_close(ticket, account, 0.0, result.price)` with a hard-coded 0.0 P&L 'placeholder'. `update_trade_close` sets `status='closed'` and `pnl=0.0`. The only mechanism that would backfill the real broker P&L — the history-sync loop — filters candidate tickets via `get_open_trade_tickets_for_account`, which selects only `status='opened'` rows (`db.py:363`), and `_sync_history_for_account` skips any closing deal whose ticket is not in that set (`executor.py:484-486`). Once the dashboard close flips status to 'closed', the trade is invisible to reconciliation forever, so every dashboard-closed trade shows `P&L = $0` in `/history` and pollutes every analytics aggregate (`win_rate`, `profit_factor`, `net_pnl` in `db.py:596-796` all read `trades.pnl` for `status='closed'`). The dashboard state permanently disagrees with the broker.

Evidence. `api/actions.py:83-85` ``if result.success: # Verbatim DB write (dashboard.py:1093): pnl 0.0 placeholder... await db.update_trade_close(ticket, account, 0.0, result.price)``; `db.py:345-353` ``UPDATE trades SET status='closed', pnl=$1, ...``; `db.py:362-366` ``SELECT ticket FROM trades WHERE account_name=$1 AND status='opened'``; `executor.py:484-486` ``ticket = deal.position_id; if ticket not in open_tickets: continue``.

FIX — Fetch the position's live profit before closing (`connector.get_positions()` already returns `pos.profit`) and write that, or better: do NOT mark the trade closed from the dashboard route — leave `status='opened'` (or a 'closing' status the history-sync also scans) and let the history-sync loop reconcile the authoritative deal profit; alternatively have the close route fetch the closing deal from `get_history_deals` immediately after close and write its profit.

Adversarial verdict: CONFIRMED. Every citation verifies verbatim: `api/actions.py:84` writes `db.update_trade_close(ticket, account, 0.0, result.price)` on successful dashboard close; `db.py:345-353` sets `status='closed'`, `pnl=0.0`; the only backfill mechanism (history-sync) selects candidates via `db.py:362-366` (`status='opened'` only) and `executor.py:484-486` skips deals whose `position_id` is not in that set, while `executor.py:506-509` advances the watermark past the closing deal so it can never be re-seen. Refutation attempts failed: `bot.py:213` backfills real `pnl` but only under ``if backend == "dry_run"`` (`bot.py:206`); `trade_manager.py:1112` writes `pos.profit` but only on the Telegram-signal close path; legacy `dashboard.py:1093` has the identical 0.0 bug; the race where the sync straddles the close cannot reliably rescue because the sync snapshots ``now`` and `open_tickets` at iteration start. Impact confirmed: analytics at `db.py:602-618/632-648/673-696` aggregate `trades.pnl` for `status='closed'`, and `pnl=0.0` counts as a loss (`pnl <= 0`), permanently skewing `win_rate`, `profit_factor`, and `net_pnl` for every dashboard full-close.

Verifier correction/refinement. Finding is accurate as described. Severity should be high, not critical: the broker-side close executes correctly and no money is lost or trading blocked — the defect is a deterministic, permanent corruption of recorded P&L and all downstream analytics on

every dashboard full-close. Fix note: writing pos.profit captured just before close (as trade_manager.py:1112 already does) records floating profit, which can differ slightly from the final deal profit; the exact-value fix is to leave the row in a status the history-sync scans (e.g. keep 'opened' or add 'closing' to get_open_trade_tickets_for_account) so executor.py:488 reconciles the authoritative deal.profit, or fetch the closing deal via get_history_deals immediately after close.

A.3 Medium/low findings (verbatim)

MEDIUM trade_manager.py:418 · domain: exec-tp-sl

A.3.1 Stage-1 align explicitly sends tp=0.0 when the follow-up has no numeric TP — clears/never sets TP and the watchdog is already disabled

Description. In _handle_correlated_followup, new_tp is initialized to 0.0 and only overwritten if signal.target_tp is truthy. A follow-up whose TP lines are all 'open' ('TP: open') parses to tps=['open'] and target_tp=None (signal_parser.py:113-120 returns None when no numeric TP), so modify_position(ticket, sl=new_sl, tp=0.0) is sent. Since 0.0 is not None, the server sets tp=0.0 (server.py:372 'req.tp if req.tp is not None else pos.tp'), which in MT5 REMOVES any TP. Because the pairing created sibling rows, the orphan protective-TP watchdog no longer applies (db.py:1134-1138), so this position permanently has no TP and rides its default SL — another concrete instance of symptom 1 for 'TP open' signals.

Evidence. trade_manager.py:418-425 'new_tp = 0.0; if signal.target_tp: new_tp = calculate_tp_with_jitter(...); ... modify_position(stage1["mt5_ticket"], sl=new_sl, tp=new_tp)'; signal_parser.py:113-120 _select_target_tp returns None when numeric list empty; server.py:371-373.

FIX — Pass tp=None (leave unchanged) instead of 0.0 when signal.target_tp is None, and route no-TP paired stage-1 positions into the protective-TP watchdog (drop the blanket sibling exclusion or add a 'tp==0' sweep as in finding 3).

A.3.2 SL-less correlated follow-up raises `TypeError` before band creation (jitter>0) or fails every band via D-08 (jitter=0)

Description. `_handle_open` has an explicit guard for `signal.sl` is `None` (trade_manager.py:564-577), but `_handle_correlated_followup` has none — `handle_signal` routes a paired OPEN there first (trade_manager.py:234-242). With a filled stage-1, it calls `calculate_sl_with_jitter(signal.sl=None, jitter_points, direction)` at line 415-417, OUTSIDE the try that only wraps the modify (line 423). If `cfg.sl_tp_jitter_points > 0`, `risk_calculator.py:75-76` evaluates '`sl + offset`' -> `TypeError` on `None`, which propagates out through `handle_signal` and `Executor._execute_single_account` (no try at `executor.py:136`), aborting processing for that account (no bands, no align, unhandled in the executor loop). If jitter is 0, `new_sl=None` reaches modify (server keeps old SL — benign), but every band later fails the D-08 backstop (trade_manager.py:914-917) or hits `calculate_sl_distance(entry, None)` `TypeError` at line 868 first.

Evidence. trade_manager.py:415-417 '`new_sl = calculate_sl_with_jitter(signal.sl, ...)`' outside the try beginning at 423; risk_calculator.py:73-76 '`if jitter_points <= 0: return sl; offset = random.uniform(...); return round(sl + offset, 2)`'; contrast the guard at trade_manager.py:564-577.

FIX — Add the same `signal.sl` is `None` guard at the top of `_handle_correlated_followup` (skip align/bands or align SL-unchanged with tp only), mirroring the EXEC2-04/D2-13 guard in `_handle_open`.

MEDIUM mt5-rest-server/server.py:415 · domain: exec-tp-sl

A.3.3 close_position hardcodes ORDER_FILLING_IOC while open path negotiates filling mode — closes rejected on non-IOC brokers

Description. create_order deliberately selects the filling mode from symbol_info.filling_mode because 'Market-execution brokers (e.g. Vantage) typically reject IOC' (server.py:276-290). close_position ignores that and hardcodes 'type_filling': mt5.ORDER_FILLING_IOC (server.py:415). On a broker/symbol that only supports FOK or RETURN, every close (signal CLOSE, partial close, kill-switch emergency_close) fails with an unsupported-filling retcode (10030) — positions can be OPENED (open path picks FOK/RETURN) but not CLOSED by the bot.

Evidence. server.py:279-290 dynamic filling selection with the Vantage comment; server.py:405-416 close request dict with '"type_filling": mt5.ORDER_FILLING_IOC' and no symbol_info consultation.

FIX — Reuse the same filling-mode negotiation (bitmask check on symbol_info.filling_mode) in close_position that create_order uses for TRADE_ACTION_DEAL.

MEDIUM trade_manager.py:1269 · domain: exec-tp-sl

A.3.4 cleanup_expired_orders filled-detection can never match — filled limit orders are marked cancelled in the DB

Description. REL-04 checks whether an expired pending order actually filled by testing 'str(order["ticket"]) in p.comment' over open positions. But a position created from a filled pending order carries the ORDER'S CLIENT COMMENT ('telebot' or 'telebot-{signal_id}-s{n}' — set at trade_manager.py:938 and passed through open_order), never the broker ticket number. So the filled branch is unreachable in practice: a limit order that filled and then aged past expiry is marked cancelled via db.mark_pending_cancelled (trade_manager.py:1284), its trades row stays status='pending', and downstream reconciliation (history-sync keyed on open trades) and analytics misclassify a live position.

Evidence. trade_manager.py:1266-1272 'filled = any(p.comment and str(order["ticket"]) in p.comment for p in positions)'; comment origin trade_manager.py:938 'target_comment = f"telebot-{signal_id}-s{stage_number}" if staged else "telebot"; server position comment passthrough server.py:246.

FIX — Match filled pending orders by comment equality (the mt5_comment is unique) or by querying history deals/orders for the order ticket, instead of substring-searching the broker ticket in the comment.

A.3.5 Zone-watch snapshot rebuild requires every AccountSettings field — schema drift silently downgrades fixed_lot stages to percent sizing

Description. `_fire_zone_stage` rebuilds the frozen snapshot with `'{f.name: snapshot_dict[f.name] for f in _dc_fields(AccountSettings))'` — a plain `KeyError` if the persisted JSONB predates any newly added `AccountSettings` field. The except sets `snapshot=None`, and `_execute_open_on_account` then takes the percent-mode branch (`trade_manager.py:874` 'if snapshot is not None and snapshot.risk_mode == "fixed_lot"' is False), sizing the stage from `acct.risk_percent` instead of the operator's fixed lot, and `default_sl_pips` falls back to 100 (`executor.py:1008`). A silently wrong lot size on a live stage after any settings-model migration.

Evidence. `executor.py:989-998` `'snapshot = AccountSettings(**{f.name: snapshot_dict[f.name] for f in _dc_fields(AccountSettings))'` with `'except Exception: ... snapshot = None'`; `trade_manager.py:874-896` sizing branch selection on snapshot presence.

FIX — Use `snapshot_dict.get(f.name, default)` with dataclass field defaults (or `AccountSettings(**{k: v for k, v in snapshot_dict.items() if k in field_names})`), and treat a rebuild failure as a hard stage failure rather than falling back to different risk semantics.

A.3.6 MODIFY_SL/MODIFY_TP/CLOSE signals act on ALL symbol positions, including manual/foreign ones; retcode 10025 no-change modifies reported as failures

Description. Two hygiene items: (1) `_handle_close` (trade_manager.py:1107-1109), `_handle_close_partial` (:1142-1149), `_handle_modify_sl` (:1179-1195) and `_handle_modify_tp` (:1225-1236) iterate `connector.get_positions(symbol)` with no magic-number or comment filter, so a channel signal closes/modifies positions the operator opened manually in the same symbol. The magic number exists (mt5_connector.py:93, sent on open at server.py:301) but is never used for selection — the positions endpoint doesn't even return it (server.py:236-248). (2) The REST modify endpoint treats every retcode `!= TRADE_RETCODE_DONE` as failure (server.py:379-380), including 10025 `TRADE_RETCODE_NO_CHANGES`, so an idempotent re-issue (e.g. duplicate breakeven signal, or a retry after a timeout whose first attempt succeeded) surfaces as a spurious `MODIFY_FAILED` error to the operator.

Evidence. trade_manager.py:1107-1109 'positions = await connector.get_positions(signal.symbol); for pos in positions: result = await connector.close_position(pos.ticket)'; server.py:236-248 position serialization lacks magic; server.py:379-380 retcode check.

FIX — Filter positions by magic (add magic to the /positions payload) or by the 'telebot' comment prefix before close/modify; on the server, treat retcode 10025 (`NO_CHANGES`) as success for `TRADE_ACTION_SLTP`.

A.3.7 Sessions are non-revocable signed cookies with rolling 30-day expiry — logout does not invalidate, no absolute lifetime, no kill-switch

Description. Auth uses Starlette SessionMiddleware pure signed-cookie sessions: there is no server-side session store, so (a) logout (dashboard.py:215-220 and api/auth.py:144 `request.session.clear()`) only clears the cookie on the responding client — a previously captured telebot_session cookie value remains cryptographically valid and replayable; (b) the installed SessionMiddleware re-signs and re-sets the cookie with `Max-Age=30d` on EVERY response with a non-empty session (`.venv starlette/middleware/sessions.py send_wrapper: 'if scope["session"]:` ... `headers.append("Set-Cookie", ...)`' with `TimestampSigner`), so an actively-used session never expires — the 30-day `max_age` is rolling, not absolute; (c) there is no way to revoke a leaked cookie for this money-moving dashboard short of rotating `SESSION_SECRET` (which logs out everyone). Failure scenario: session cookie exfiltrated once (shared machine, device theft) grants indefinite trade-management access with no operator-side revocation.

Evidence. dashboard.py:179-187 `SessionMiddleware(secret_key=..., max_age=30*24*60*60);` dashboard.py:215-220 logout only does ``request.session.clear()``; api/auth.py:140-145 same; starlette sessions.py (installed 0.4x) re-issues ``Set-Cookie ... Max-Age={self.max_age}`` on every non-empty-session response and validates only ``self.signer.unsign(data, max_age=self.max_age)`` — timestamp refreshed on each re-sign, hence rolling.

FIX — For a trading control panel, move to a server-side session record (sessions table in the existing Postgres: `session_id` -> `user`, `created_at`, `last_seen`, `revoked` flag), store only the opaque `session_id` in the signed cookie, enforce an absolute lifetime (e.g. 30d from `created_at`) alongside the idle timeout, and have logout delete the row. This also gives per-device session listing/revocation for free. Minimal alternative: embed an issued-at in the session dict, reject sessions older than an absolute cap in `require_user`, and document `SESSION_SECRET` rotation as the emergency revocation procedure.

A.3.8 GET /logout is CSRF-exempt — cross-site forced logout of the operator

Description. dashboard.py registers logout on both POST and GET with no CSRF guard. The session cookie is SameSite=Lax, which IS sent on top-level GET navigations, so any page the operator visits can force-log-out the dashboard with a simple link/redirect to `https://dashboard.../logout` (e.g. `<a>` click or `window.location`). Nuisance/DoS: mid-trade-management the operator is bounced to `/app/login`. Not exploitable for state change beyond session destruction.

Evidence. dashboard.py:215-220: ``@app.post("/logout")` + `@app.get("/logout")` + `async def logout(request): request.session.clear(); return RedirectResponse(url="/app/login", status_code=303)`` — no `verify_csrf_token` dependency; contrast `api/auth.py:140` where `/api/v2/auth/logout` is guarded by `Depends(_verify_csrf)`.

FIX — Delete the GET variant and the legacy `/logout` route entirely — the SPA's sign-out uses the CSRF-guarded POST `/api/v2/auth/logout` (`frontend/src/components/shell/Sidebar.tsx:59`). If a plain-link logout must stay, keep GET but render an interstitial confirm form that POSTs with the CSRF token.

A.3.9 Naked double-submit CSRF cookie without `__Host-` prefix or session binding — forgeable by anything that can set cookies for the parent domain

Description. `verify_csrf_token` compares the `telebot_csrf` cookie to the `X-CSRF-Token` header with no signing, no session binding, and no `__Host-` prefix. Classic double-submit weakness: any principal able to plant a `telebot_csrf` cookie scoped to `dashboard.YOURDOMAIN.com` — a compromised/XSS'd sibling app on another subdomain of the shared VPS nginx (Domain=YOURDOMAIN.com cookies flow down), or an active network attacker pre-HSTS-pin setting cookies over http — can choose the token value and then forge state-changing requests carrying the matching header value in a cross-site context is still blocked by `SameSite=Lax` on POST, so practical exploitability today is low (requires subdomain compromise + a Lax bypass or top-level POST vector). Defense-in-depth gap rather than an open hole. Positive notes for the audit record: comparisons use `secrets.compare_digest` (timing-safe, `api/auth.py:105`, `api/deps.py:65`), rate-limit runs before `argon2` cost (`api/auth.py:113`), secure flag is config-driven, and the CSRF cookie being `httponly=False` is a correct, deliberate double-submit requirement.

Evidence. `api/deps.py:62-66` — the entire guard is ``cookie = request.cookies.get(CSRF_COOKIE, ""); header = request.headers.get(CSRF_HEADER, ""); if not cookie or not header or not _secrets.compare_digest(cookie, header): raise HTTPException(403)``. Cookie set without `__Host-` prefix at `api/auth.py:80-87`. `nginx/telebot.conf` sets ``Strict-Transport-Security "...includeSubDomains"`` implying sibling subdomains exist on the shared VPS (MEMORY: shared nginx hosts multiple apps).

FIX — Rename the cookie to ``__Host-telebot_csrf`` (requires `Secure + Path=/ + no Domain` — all already true in prod), which prevents subdomain/network cookie planting. Stronger: bind the token to the session (store a random csrf value in `request.session` at login and compare header against it — synchronizer pattern; the session dict already exists and is signed), removing reliance on cookie integrity entirely.

A.3.10 `_client_ip` trusts X-Real-IP header unconditionally — login lockout keyed on a spoofable value for any non-nginx path to the app

Description. The per-IP 5-failures/15-min lockout (`api/auth.py:113 get_failed_login_count(ip)`) keys on `_client_ip`, which prefers the X-Real-IP request header with no trusted-proxy check. In the deployed topology this is mitigated: `docker-compose.yml` publishes no ports for the telebot service and nginx always overwrites X-Real-IP with `$remote_addr` (`nginx/telebot.conf`), so external clients cannot spoof it. But any container on the shared proxy-net network, or any future deployment that exposes 8080 directly (e.g. the dev compose files, or a LAN deployment without nginx), can send arbitrary X-Real-IP values and (a) rotate IPs to get unlimited argon2 brute-force attempts against the single admin password, or (b) not applicable externally. Latent misconfiguration trap rather than a live hole.

Evidence. `dashboard.py:117-122`: ``xri = request.headers.get("x-real-ip", "").strip(); if xri: return xri``. `api/auth.py:113`: ``if await db.get_failed_login_count(ip, minutes=15) >= 5: raise HTTPException(429)``. `docker-compose.yml` telebot service has no ``ports:`` mapping; `nginx/telebot.conf` ``proxy_set_header X-Real-IP $remote_addr;`` on both locations.

FIX — Gate the header on a trusted-proxy allowlist (only honor X-Real-IP when `request.client.host` is the nginx container / RFC1918 proxy address, configurable via env), or additionally cap total failed logins across ALL IPs in the window (e.g. global `>= 20` in 15 min -> 429) so IP-spoofing cannot fully bypass the lockout for a single-operator system.

A.3.11 Drilldown payload never carries the `_display` twins the SPA renders — fill history and signal timestamp show em-dashes for every trade

Description. `position_drilldown` only enriches three fields on the ``position`` block (`entry_price_display`, `lot_size_display`, `pnl_display` at `api/positions.py:69-74`) and returns `fill_history` and `signal raw`. The SPA component renders EXCLUSIVELY the `*_display` twins for the fill-history table and signal timestamp: `PositionDrilldown.tsx` declares `filled_at_display`, `lot_size_display`, `band_low_display`, `band_high_display`, `sl_at_fill_display` (lines 36-43) and `timestamp_display` (line 50), and every cell falls back to `"—"` when absent (lines 110-130). None of those keys are ever produced by the backend, so once the 500 in finding 1 is fixed, the entire fill-history table and the signal timestamp render as dashes for all trades — the details view is effectively empty. Additionally the raw ``filled_at`/`timestamp`` datetimes are serialized by FastAPI's default encoder, bypassing the D-05/D-06 `ts_machine/ts_display` contract used by every other route.

Evidence. `api/positions.py:66-75` enriches only `pos["entry_price"]`/`["lot_size"]`/`["pnl"]` then ``return detail`` — `fill_history/signal` untouched. `db.py:1285-1293` builds `fill_history` rows with bare keys (`stage_number`, `filled_at`, `lot_size`, `band_low`, `band_high`, `sl_at_fill`, `status`).
`frontend/src/components/positions/PositionDrilldown.tsx:110` ``cell: (f) => f.filled_at_display ??``
`"—"`, :116 `f.lot_size_display ?? "—"`, :123-124 `f.band_low_display && f.band_high_display ? ...``
`: (implicit dash)`, :130 `f.sl_at_fill_display ?? "—"`.`

FIX — In `position_drilldown`, enrich each `fill_history` row via `api/formatting.py` (`volume_display` for `lot_size`, `price_display(symbol, band_low/band_high)`, `ts_machine/ts_display` pair for `filled_at`) and the signal block's timestamp, matching the `FillRow/DrilldownSignal` interfaces in `PositionDrilldown.tsx`.

A.3.12 sl_at_fill mixes units: pips for staged fills, absolute price for single-stage fills

Description. In `get_position_drilldown`, the staged branch populates ``sl_at_fill`` with ``settings.get("default_sl_pips")`` — a pip count like 100 — while the single-stage fallback populates the same field with ``trade["sl"]`` — an absolute price like 2350.50 (db.py:1303). The same column in the drilldown fill-history table therefore has incompatible semantics depending on trade type; any future `price_display` formatting of it (per finding 2's fix) would print nonsense for staged rows.

Evidence. db.py:1291 ``"sl_at_fill": settings.get("default_sl_pips"),`` vs db.py:1303 ``"sl_at_fill": trade["sl"],``

FIX — Store one semantic: either compute the actual SL price at fill for staged rows (band edge $\pm$ default_sl_pips * pip_size, as `executor.py:907` does), or rename/split into `sl_pips_at_fill` vs `sl_price_at_fill` and render each appropriately.

A.3.13 No catch-all exception handler for `/api/v2` — unhandled errors bypass the error envelope, leaving the SPA with a generic unusable message

Description. `register_error_handlers` installs handlers only for `HTTPException`, `StarletteHTTPException` and `RequestValidationError`. Any other exception raised in an `/api/v2` route (like the `AttributeError` in finding 1, or a future `asyncpg/connector` error) propagates to Starlette's default 500 with a plain-text "Internal Server Error" body — violating the module's own documented contract ("Failure shape ... `{\"error\": {\"code\", \"message\"}}`") and forcing the SPA `ErrorPanel` into its fallback string (`ErrorPanel.tsx:33` ``Request failed (HTTP ${error.status}).``), which is what the operator reports as an unexplained "unable to get ..." failure. A structured 500 envelope (plus server-side logging of the traceback) would have made symptom 3 diagnosable from the dashboard.

Evidence. `api/errors.py:85-87` registers exactly three handlers:

```
`app.add_exception_handler(HTTPException, _http_exception_handler`,  
`app.add_exception_handler(StarletteHTTPException, _http_exception_handler`,  
`app.add_exception_handler(RequestValidationError, _validation_exception_handler)` — no  
`app.add_exception_handler(Exception, ...)`.
```

FIX — Add a generic Exception handler that, for `request.url.path.startswith("/api/v2")`, logs the traceback and returns `JSONResponse(500, _envelope("internal", "Internal server error"))`; leave non-API paths on the default behaviour.

A.3.14 Expired-limit-order fill detection matches str(ticket) against position comment, but comments never contain the ticket — every filled-then-expired limit is misclassified as cancelled

Description. cleanup_expired_orders, when an expired order is gone from MT5's pending list, decides 'filled vs already-removed' by checking whether the order ticket appears in any open position's comment. But positions inherit the ORDER's comment, which this codebase sets to 'telebot' or 'telebot-{signal_id}-s{n}' — never the ticket number. The check is structurally always False, so a limit order that FILLED just before expiry is marked cancelled in the DB (mark_pending_cancelled) while a real position rides on the broker; the trades row stays status='pending' and the position is invisible to close-signal bookkeeping and analytics until (if ever) the history sync catches its close.

Evidence. trade_manager.py:1268-1272: ``filled = any(p.comment and str(order["ticket"]) in p.comment for p in positions)``. Comments are set at trade_manager.py:938: ``target_comment = f"telebot-{signal_id}-s{stage_number}" if staged else "telebot"`` — no ticket component. In MT5, a filled pending order produces a position whose ticket EQUALS the order ticket, which is the correct join key.

FIX — Replace the comment substring check with ``filled = any(p.ticket == order["ticket"] for p in positions)`` (position ticket == originating order ticket for pending fills), and on fill also flip the trades row to status='opened' with the fill price.

A.3.15 No durable order intent before broker submission on the unstaged path, and no reconciliation loop that adopts unknown broker positions

Description. Professional executors persist an order-intent record BEFORE `order_send` and reconcile broker state against it on restart. The staged path approximates this (`staged_entries` row inserted pre-submit, comment-based reconcile on reconnect at `executor.py:246-300`). But the v1.0 unstaged path (`_handle_open` degenerate no-zone branch, line 610-622) writes `db.log_trade` only AFTER the order result returns: a crash/kill between `order_send` success and `log_trade` leaves a live position with zero DB record. Its comment is the literal shared string 'telebot' (`trade_manager.py:938`), so even a comment-based reconcile could never map it to a signal. There is also no periodic full reconciliation comparing DB 'opened' trades against broker positions in the forward direction (broker position with no DB row) — `_sync_positions` only reconciles `staged_entries` rows, and `_history_sync_loop` only reconciles CLOSSES of already-known tickets (`executor.py:484-485`: ``if ticket not in open_tickets: continue``).

Evidence. `trade_manager.py:960-1016`: `order_send` at 966/979, ``await db.log_trade(...)`` only at 1004 inside ``if result.success``. `executor.py:246-254`: reconcile fetches only ``db.get_pending_stages(...)``. `executor.py:484-485`: history sync skips any deal whose `position_id` is not already in `open_tickets` — an unknown position is never surfaced.

FIX — Insert a trades row with `status='submitting'` (including the unique comment) before `open_order`, update it to `opened/failed` after; add a periodic reconciliation pass that lists broker positions with the bot's magic number and flags/adopts any without a DB row, alerting the operator.

A.3.16 Correlated follow-up with sl=None crashes with TypeError — the D2-13 SL-less guard exists only in `_handle_open`, not in `_handle_correlated_followup`

Description. `_handle_open` explicitly guards `signal.sl` is `None` ('would crash at `calculate_sl_distance(entry, None)`'). `_handle_correlated_followup` has no such guard: it only validates `entry_zone` and `direction` (line 382). A follow-up OPEN that pairs to an orphan but carries no SL reaches `calculate_sl_with_jitter(None, jitter, dir)` — with `sl_tp_jitter_points > 0` that is ``round(None + offset, 2)`` → `TypeError` at line 415-417 (outside the try that only wraps the modify call); with jitter 0 it survives to `calculate_sl_distance(entry_for_calc, None)` at line 868 → `TypeError`. Either way the exception propagates out of `temp_tm.handle_signal` and aborts the account's entire follow-up processing (no bands created, no align), unhandled in `Executor.execute_signal`.

Evidence. Guard exists only at `trade_manager.py:557-577`: `'# A standalone OPEN whose signal.sl is None would crash at calculate_sl_distance(entry, None) ...'`. `_handle_correlated_followup` entry check at 382: ``if signal.entry_zone is None or signal.direction is None:`` — no sl check. `risk_calculator.py:73-76`: ``if jitter_points <= 0: return sl; offset = random.uniform(...); return round(sl + offset, 2)`` — `None + float` raises. `risk_calculator.py:89`: ``return abs(entry_price - sl)``.

FIX — Add the same `sl-is-None` guard at the top of `_handle_correlated_followup` (skip the align, use the stage-row NULL-fallback path that `_fire_zone_stage` already implements at `executor.py:1026-1028`), and wrap the per-account body in the same failure-isolation pattern used elsewhere.

A.3.17 Point-band stages (`zone_low == zone_high`) can never fire through the zone watcher — pre-flight tolerance degenerates to zero

Description. `compute_bands` explicitly supports `zone_low == zone_high` ('Research Q5 RESOLVED: ... point-bands, which fire together on arrival'). But in `_zone_watch_loop` the D-14 pre-flight computes `band_width = 0`, `tolerance = 0.5*0 = 0`, then requires ``abs(mid_price - price_center) > band_width + tolerance`` to NOT hold — i.e. `mid_price` must EXACTLY equal the point. Since `mid = (bid+ask)/2` and the in-zone trigger for a buy is `ask <= band.high`, `mid < ask <= center` strictly (nonzero spread), the pre-flight always rejects: armed point-band stages are skipped on every tick forever, silently (only a 'price drifted' info log each 10s). Only bands already in-zone at signal arrival ever fill for point zones.

Evidence. `trade_manager.py:80-81`: ``if zone_low == zone_high: return [Band(stage_number=i + 2, low=zone_low, high=zone_low) ...]``. `executor.py:688-690`: ``band_width = max(band_high - band_low, 0.0); tolerance = 0.5 * band_width``. `executor.py:749-757`: ``if abs(mid_price - price_center) > band_width + tolerance: logger.info(... "price drifted") continue`` — with `band_width=0` this is ``abs(mid - center) > 0``, true whenever `spread > 0`.

FIX — Floor the pre-flight tolerance at a symbol-appropriate minimum (e.g. ``max(band_width + tolerance, k * pip_size)`` or the current spread), so point/narrow bands remain fireable while wide-drift protection is preserved.

A.3.18 Float arithmetic and hardcoded rounding throughout price/lot handling — no Decimal, no broker volume_step/digits awareness; fixed_lot per-stage floor can exceed the configured total

Description. All prices and lots are binary floats. Lot sizes are `round(lot_size, 2)`, silently assuming `volume_step=0.01` and `min 0.01` — never read from `symbol_info` (server queries `symbol_info` only for `filling_mode`, `server.py:279`). A broker with `volume_step 0.1` gets `retcode 10014` (invalid volume) on e.g. `0.15` lots — which the no-taxonomy error path (finding 6) then reports as an opaque failure. SL/TP jitter rounds to 2 decimals hardcoded (fine for XAUUSD, wrong for 3/5-digit symbols the pip-size fallbacks at `trade_manager.py:120-125` and `executor.py:1007` nominally allow). Also, `fixed_lot` mode floors each stage at `0.01` (`max(0.01, min(stage_lot_size(snapshot), max_lot))`, `trade_manager.py:876`), so `risk_value=0.02` with `max_stages=4` deploys `0.04` total — exceeding the operator's configured TOTAL (which per project memory is defined as total across `max_stages`). Standard practice is Decimal (or integer points/steps) with symbol digits and `volume_step` quantization fetched from the broker.

Evidence. `risk_calculator.py:64` `return round(lot_size, 2)`; `:62` `lot_size = max(lot_size, 0.01)`; `:76/:84` `return round(sl + offset, 2)`. `trade_manager.py:876`: `lot_size = round(max(0.01, min(stage_lot_size(snapshot), max_lot)), 2)`. `server.py` never sends `volume_min/volume_step/digits` to the client.

FIX — Expose `volume_min/volume_step/digits` via a `/api/v1/symbol/{symbol}` endpoint; quantize lots to `volume_step` and prices to digits at the connector boundary; drop the unconditional per-stage `0.01` floor in `fixed_lot` mode (skip the stage instead of inflating total risk) or warn when floor inflation occurs.

A.3.19 Structured broker error detail is discarded at the connector boundary — modify/close failures surface as generic 'REST API request failed'

Description. The server returns rich errors for modify/close (e.g. 'retcode=10016 Invalid stops') via HTTPException detail. `_request` unwraps the detail, LOGS the code/message, but then returns `None` — so `modify_position/close_position` build `OrderResult(error="REST API request failed")`. Callers that branch or report on `result.error` (stage-1 align warning at `trade_manager.py:463-465`, orphan watchdog at `executor.py:956-959`, kill-switch close log at `executor.py:343`) all record the useless generic string. For symptom 1 specifically, this destroys the evidence trail of WHY a TP attach failed, forcing log archaeology across two hosts.

Evidence. `mt5_connector.py:601-606`: ``if not body.get("ok"): error = body.get("error") or {};`
`logger.warning(...); return None`` — error content dropped. `mt5_connector.py:736-737`: ``if data is`
`None: return OrderResult(success=False, ticket=ticket, error="REST API request failed")``.

FIX — Have `_request` return a (data, error) tuple or raise a typed exception carrying `error.code/error.message`; propagate it into `OrderResult.error` so `retcode`-level causes appear in notifications and DB `cancelled_reason` fields.

A.3.20 `update_stage_status('filled')` has no status precondition — kill-switch-cancelled stage can be resurrected by an in-flight fill

Description. `emergency_close` sets `_trading_paused` then drains all pending staged_entries rows to 'cancelled_by_kill_switch' BEFORE closing positions (`executor.py:311-322`). But an `_execute_open_on_account` call already past its guards (the pause is only re-checked inside the zone-watch loop, `executor.py:684/760` — TradeManager itself never checks it) may be awaiting `connector.open_order` when the drain runs. When the order returns, `trade_manager.py:1003` calls `update_stage_status(stage_row_id, 'filled', ...)`, whose UPDATE has no WHERE status guard, overwriting the terminal 'cancelled_by_kill_switch'. Meanwhile `emergency_close`'s `get_positions` snapshot (`executor.py:335`) can miss the still-landing order, so a live position survives 'close all positions' while the DB claims the kill switch drained it. The same missing precondition lets 'failed' writes from `executor.py:1086-1090` clobber 'filled' set milliseconds earlier by a concurrent path (see the duplicate-fire race).

Evidence. `db.py:1055-1069` ``UPDATE staged_entries SET status=$1, mt5_ticket=$2, filled_at=NOW() WHERE id=$3`` — id-only predicate; `executor.py:311` ``self._trading_paused = True`` then `318` ``drained_stages = await db.drain_staged_entries_for_kill_switch()`` then `335` ``positions = await connector.get_positions()``; `trade_manager.py:1002-1003` unconditional ``await db.update_stage_status(stage_row_id, "filled", mt5_ticket=result.ticket)`` with no pause check anywhere in `_execute_open_on_account`.

FIX — Make stage status transitions compare-and-set: ``... WHERE id=$3 AND status IN ('awaiting_zone','awaiting_followup')`` and have callers inspect the affected-row count; if the fill lands on an already-cancelled row, immediately close the just-opened ticket (kill-switch semantics) or at minimum alert. Also add a `_trading_paused` re-check in TradeManager immediately before `connector.open_order`.

A.3.21 Limit-order trades are logged status='pending' and never transition — broker close of a limit-filled position is never reconciled

Description. When a zone entry places a limit order, `log_trade` writes the trades row with `status='pending'`. When the order later fills, `cleanup_expired_orders` (trade_manager.py:1273-1274) and the dry-run callback (bot.py:222) call only `db.mark_pending_filled`, which updates the `pending_orders` table — nothing ever flips `trades.status` from 'pending' to 'opened'. The history-sync loop selects reconciliation candidates with ``WHERE status='opened'`` (db.py:363), so when the broker eventually closes that position via SL/TP, the close is never written back: the trade shows 'pending' in /history forever with `pnl=0`, and analytics (`status='closed'` filters) never count it. Additionally the recorded ticket is the ORDER ticket, which on MT5 differs from the resulting `position_id` used by `deal.position_id` matching, so even a status fix needs the ticket mapped at fill time.

Evidence. trade_manager.py:1014 ``status="opened" if use_market else "pending"``; db.py:468-474 `mark_pending_filled` updates only `pending_orders`; `grep` shows the only `trades-status` write is `update_trade_close`'s ``SET status='closed'`` (db.py:346); db.py:363 ``SELECT ticket FROM trades WHERE account_name=$1 AND status='opened'``.

FIX — When a pending order is detected as filled (`cleanup_expired_orders`' filled branch and any fill detection path), also ``UPDATE trades SET status='opened', ticket=<position_ticket>, entry_price=<fill price> WHERE ticket=<order_ticket> AND account_name=$2 AND status='pending'``, capturing the position ticket from the matched MT5 position.

A.3.22 No startup reconcile: D-24 staged-entry/position reconciliation only runs on heartbeat reconnect, never on process restart

Description. On startup, bot.py connects each account with `connector.connect()` directly and starts the executor; `_sync_positions` (the D-24 reconcile that marks `awaiting_zone` rows 'filled' when a matching comment is live on MT5, or 'abandoned_reconnect' when stale) is invoked ONLY from `Executor._reconnect_account` (`executor.py:213`). If the bot crashes between `connector.open_order` returning and `update_stage_status('filled')` (or is restarted with armed stages), the DB row stays 'awaiting_zone' while a live position exists on the broker. The zone-watch loop cannot repair this unless price re-enters the band: the band gate ``if not stage_is_in_zone_at_arrival(...): continue`` (`executor.py:698`) runs BEFORE the D-25 idempotency probe (`executor.py:764-775`), so an out-of-band filled-but-unrecorded stage is never marked filled and `/stages` shows a phantom armed stage indefinitely — dashboard state disagreeing with the broker. It can also stay eligible to 'fire' again much later if price revisits the band long after the signal is dead (only the D-25 comment probe then saves it from double-submit, provided the position is still open; if the position was already closed by the broker, the probe misses and the stage re-fires a stale entry).

Evidence. `bot.py:276-283` startup loop calls only ``connected = await asyncio.wait_for(connector.connect(), timeout=15)`` — no reconcile; `executor.py:210-214` ``success = await asyncio.wait_for(connector.connect(), timeout=15); if success: await self._sync_positions(acct_name, connector)`` exists only in `_reconnect_account`; `executor.py:698` ``if not stage_is_in_zone_at_arrival(band, bid, ask, direction): continue`` precedes the D-25 probe at `executor.py:763-775`.

FIX — Run `_sync_positions(acct_name, connector)` for every successfully connected account at startup (after `executor.start()` or inside the bot.py connect loop), and add an age cutoff in `_zone_watch_loop` that abandons `awaiting_zone` rows older than `signal_max_age_minutes` even when price never re-enters the band.

A.3.23 Daily trade-limit check is read-then-act with concurrent Telethon handlers — limit can be exceeded

Description. TelegramClient is constructed without sequential_updates, so Telethon dispatches each incoming message handler as its own task; two signals arriving in quick succession run executor.execute_signal concurrently. _execute_open_on_account enforces the daily cap by reading get_daily_stat(name, 'trades_count') and comparing (trade_manager.py:794-798) but only increments after the order succeeds (trade_manager.py:989-1000) — a classic unlocked read-modify-write. At trades_count == limit-1, both concurrent signals read limit-1, both pass, both execute → cap exceeded. The same unlocked pattern applies to max_daily_server_messages (line 800-804). The staggered per-account delays widen the window to multiple seconds.

Evidence. bot.py:248-255 `TelegramClient(StringSession(...), ..., connection\_retries=10, retry\_delay=5, auto\_reconnect=True)` — no sequential_updates=True; trade_manager.py:794-798 `trade\_count = await db.get\_daily\_stat(name, "trades\_count"); if trade\_count >= self.cfg.max\_daily\_trades\_per\_account: ...`; increment deferred to trade_manager.py:995-1000.

FIX — Enforce the cap atomically in SQL: make increment_daily_stat conditional (`UPDATE ... SET trades\_count = trades\_count + 1 WHERE ... AND trades\_count < \$limit RETURNING trades\_count`) and reserve the slot BEFORE submitting the order (releasing it on failure), or serialize signal execution with a per-account asyncio.Lock in Executor.